فصل ۱۸: معماری میکروسرویس‌ها

معماری میکروسرویس‌ها `(microservices architecture)` سبکی بسیار محبوب است که در سال‌های اخیر شتاب چشمگیری گرفته است. در این فصل، ویژگی‌های مهمی را مرور می‌کنیم که این معماری را، هم از نظر توپولوژی و هم از نظر فلسفه، از دیگر سبک‌ها متمایز می‌کنند.

بیشتر سبک‌های معماری پس از پیدایش‌شان نام می‌گیرند؛ معمارانی متوجه می‌شوند که الگوی خاصی مدام تکرار می‌شود و برایش نامی برمی‌گزینند. هیچ گروه مخفیِ معمارانی وجود ندارد که تصمیم بگیرد جنبش بزرگ بعدی چه باشد. معماران همگام با جابه‌جایی‌ها و دگرگونی‌های اکوسیستم توسعهٔ نرم‌افزار تصمیم می‌گیرند و از میان رایج‌ترین شیوه‌های کنار آمدن با این تحولات و بهره‌بردن از آن‌ها، شیوه‌هایی که بهترین از آب درمی‌آیند به سبک‌های معماری‌ای تبدیل می‌شوند که دیگران از آن‌ها پیروی می‌کنند.

سبک میکروسرویس‌ها از این نظر متفاوت است: این سبک نسبتاً زود و در همان آغاز کاربردش نام گرفت. Martin Fowler و James Lewis آن را در یک پست وبلاگیِ مشهور در سال ۲۰۱۴ رواج دادند؛ آن‌ها در این پست ویژگی‌های این سبک معماریِ نسبتاً نو را بازشناختند و ترسیم کردند. پست وبلاگ آن‌ها تعریف این معماری را شکل داد و به معماران کنجکاو کمک کرد فلسفهٔ زیربنایی آن را درک کنند.

سبک میکروسرویس‌ها به‌شدت از ایده‌های طراحی حوزه‌محور (domain-driven design یا `DDD`) الهام گرفته است؛ فرایندی برای طراحیِ منطقیِ پروژه‌های نرم‌افزاری. به‌ویژه یکی از مفاهیم `DDD`، یعنی زمینهٔ محدود `(bounded context)`، الهام‌بخش قاطعِ میکروسرویس‌ها بوده است. مفهوم زمینهٔ محدود نمایانگر نوعی سبکِ کاهش کوپلینگ است (چنان‌که پیش‌تر در بخش «زمینهٔ محدود در طراحی حوزه‌محور» در فصل ۷ دیدیم)، و به همین دلیل میکروسرویس‌ها را گاهی معماری «بدون اشتراک» `(share nothing)` می‌نامند.

وقتی توسعه‌دهنده‌ای حوزه‌ای را تعریف می‌کند، آن حوزه موجودیت‌ها و رفتارهای بسیاری را در بر می‌گیرد که در مصنوعاتی مانند کد و طرح‌واره‌های پایگاه‌داده شناسایی می‌شوند. برای مثال، یک برنامهٔ کاربردی ممکن است حوزه‌ای به نام `CatalogCheckout` داشته باشد که مفاهیمی مانند اقلام کاتالوگ، مشتریان و پرداخت را شامل می‌شود. در یک معماری تک‌پارچهٔ سنتی، توسعه‌دهندگان بسیاری از این مفاهیم را به اشتراک می‌گذارند و کلاس‌های قابل بازاستفاده و پایگاه‌داده‌های به‌هم‌پیوسته می‌سازند. درون یک زمینهٔ محدود، اجزای داخلی مانند کد و طرح‌واره‌های داده می‌توانند برای انجام کار با هم کوپل شوند، اما **هرگز** به چیزی بیرون از آن زمینهٔ محدود، مانند پایگاه‌داده یا تعریف کلاسی از زمینهٔ محدودِ دیگر، کوپل نمی‌شوند. این کار به هر زمینه اجازه می‌دهد فقط آنچه را خودش نیاز دارد تعریف کند و ناچار نباشد نیاز دیگر ذی‌نفعان را هم برآورده کند؛ در نتیجه، بازاستفاده میان زمینه‌های محدود کاهش می‌یابد.

بازاستفاده در حالت کلی سودمند است، اما قانون اول معماری نرم‌افزار `(First Law of Software Architecture)` را به یاد دارید؟ هرچه هست، یک مصالحه است. عیب بازاستفاده این است که دستیابی به آن معمولاً مستلزم افزایش کوپلینگ سیستم است؛ چه از راه وراثت `(inheritance)` و چه از راه ترکیب `(composition)`.

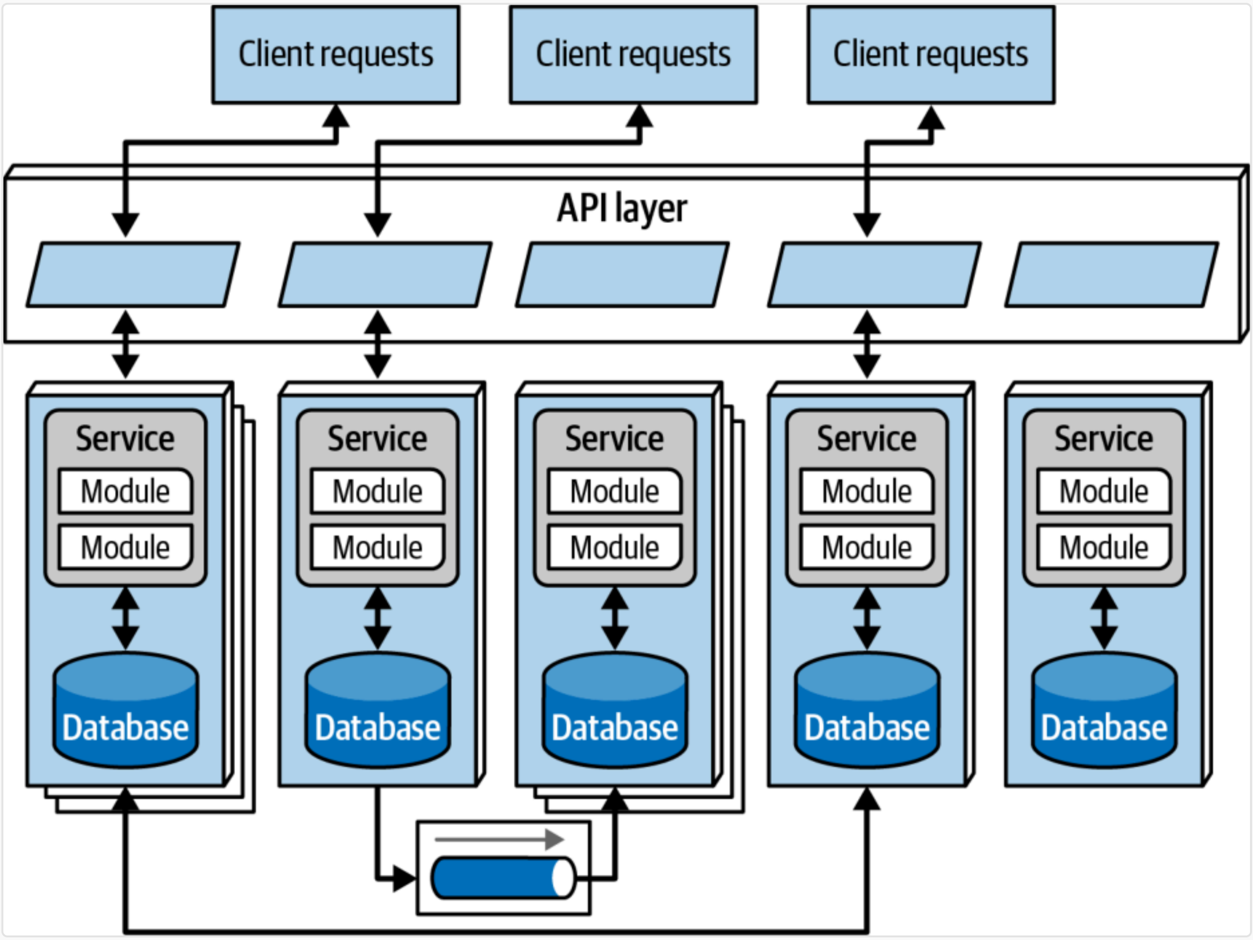
اگر هدف معمار سیستمی با کوپلینگ بسیار کم باشد (که هدف اصلی میکروسرویس‌هاست)، دوباره‌نویسی `(duplication)` را بر بازاستفاده ترجیح می‌دهد و مفهوم منطقیِ زمینهٔ محدود را به‌صورت فیزیکی مدل می‌کند، به گونه‌ای که یک سرویس و داده‌های متناظرش را در بر بگیرد.

توپولوژی

توپولوژی پایهٔ میکروسرویس‌ها در شکل ۱۸-۱ نشان داده شده است. سرویس‌ها در این سبک معماری به دلیل ماهیت تک‌منظوره‌شان بسیار کوچک‌تر از سرویس‌ها در دیگر معماری‌های توزیع‌شده‌اند؛ معماری‌هایی مانند `SOA` ارکستراسیون‌محور (فصل ۱۷)، معماری رویدادمحور (event-driven architecture؛ فصل ۱۵) و معماری مبتنی بر سرویس (service-based architecture؛ فصل ۱۴). معماران انتظار دارند هر سرویس همهٔ اجزای لازم برای عملکرد مستقل خود، از جمله پایگاه‌داده‌ها و دیگر مؤلفه‌های وابسته، را در خود داشته باشد.

میکروسرویس‌ها یک سبک معماری **توزیع‌شده** `(distributed architecture)` است: هر سرویس در فرایند `(process)` مخصوص به خود اجرا می‌شود، چه در یک ماشین مجازی `(virtual machine)` و چه در یک کانتینر `(container)`. جداسازیِ سرویس‌ها تا این حد، راه‌حل ساده‌ای برای مشکلی رایج در معماری‌هایی فراهم می‌کند که برای میزبانی برنامه‌های کاربردی به‌شدت به زیرساخت چندمستأجری `(multitenant infrastructure)` متکی‌اند. برای مثال، وقتی سیستم از یک سرور برنامهٔ کاربردی `(application server)` برای مدیریت چند برنامهٔ در حال اجرا استفاده می‌کند، این سرور امکان بازاستفادهٔ عملیاتی `(operational reuse)` از پهنای باند شبکه، حافظه، فضای دیسک و منابعی از این دست را فراهم می‌کند. اما اگر همهٔ برنامه‌های پشتیبانی‌شده همچنان رشد کنند، سرانجام یکی از منابعِ زیرساختِ مشترک با محدودیت روبه‌رو می‌شود.

مشکل دیگر، جداسازیِ نادرست میان برنامه‌های کاربردیِ مشترک است. جدا کردن هر سرویس در فرایند مخصوص به خود همهٔ مشکلاتی را که اشتراک پدید می‌آورد حل می‌کند. پیش از تکامل سیستم‌عامل‌های آزادِ رایگان و تأمین خودکار ماشین‌ها `(automated machine provisioning)`، اینکه هر حوزه زیرساخت مخصوص به خود را داشته باشد عملی نبود. اما اکنون، با منابع ابری و فناوری کانتینر (نگاه کنید به بخش «ملاحظات ابری» در همین فصل)، تیم‌ها می‌توانند از مزایای جداسازیِ حداکثری، هم در سطح حوزه و هم در سطح عملیاتی، بهره ببرند.


Client requests
Client requests
Client requests
API layer
Service
Module
Module
Database
Service
Module
Module
Database
Service
Module
Module
Database
Service
Module
Module
Database
Service
Module
Module
Database

شکل ۱۸-۱. توپولوژی سبک معماری میکروسرویس‌ها

کارایی اغلب نقطهٔ ضعفِ ماهیت توزیع‌شدهٔ میکروسرویس‌هاست. فراخوانی‌های شبکه‌ای `(network calls)` بسیار بیشتر از فراخوانی‌های متد `(method calls)` طول می‌کشند و بررسی‌های امنیتی در هر نقطهٔ پایانی `(endpoint)` زمان پردازش را افزایش می‌دهند؛ از این رو، معماران باید پیامدهای دانه‌بندی `(granularity)` را با دقت بسنجند.

از آنجا که میکروسرویس‌ها یک معماری توزیع‌شده است، معماران باتجربه توصیه می‌کنند از تراکنش‌هایی که از مرز سرویس‌ها عبور می‌کنند پرهیز شود. تعیین دانه‌بندی سرویس‌ها در واقع کلید موفقیت در این معماری است.

ویژگی‌های خاصِ سبک

بخش‌های پیش رو، بی‌آنکه جامع باشند، برخی از جنبه‌های مهم توپولوژی میکروسرویس‌ها را شرح می‌دهند؛ از جمله عناصری که مختص میکروسرویس‌ها هستند.

زمینهٔ محدود

بیایید عمیق‌تر به مفهوم زمینهٔ محدود بپردازیم؛ مفهومی که آن را فلسفهٔ پیشرانِ میکروسرویس‌ها خواندیم. هر سرویس کارکرد، زیرحوزه `(subdomain)` یا گردش‌کار `(workflow)` خاصی را مدل می‌کند. بنابراین، هر زمینهٔ محدود همهٔ آنچه را برای عملکرد درون آن کارکرد یا زیرحوزه لازم است در بر می‌گیرد؛ از جمله سرویس‌هایی متشکل از مؤلفه‌ها و کلاس‌های منطقی، طرح‌واره‌های پایگاه‌داده، و پایگاه‌دادهٔ متناظری که سرویس برای انجام کارکردهایش به آن نیاز دارد. هر سرویس قرار است نمایندهٔ یک زیرحوزه یا کارکرد مشخص باشد.

این فلسفه بسیاری از تصمیم‌هایی را که معماران در میکروسرویس‌ها می‌گیرند هدایت می‌کند. برای مثال، در یک سیستم تک‌پارچه، توسعه‌دهندگان معمولاً کلاس‌های مشترکی مانند `Address` را میان بخش‌های ناهمگون برنامه به اشتراک می‌گذارند. اما از آنجا که معماری‌های میکروسرویس می‌کوشند از کوپلینگ پرهیز کنند، معماری که با این سبک کار می‌کند به جای کوپلینگ، دوباره‌نویسی را برمی‌گزیند تا همهٔ کد را درون زمینهٔ محدودِ همان کارکرد یا زیرحوزه نگه دارد.

سبک میکروسرویس‌ها مفهوم معماریِ مبتنی بر افراز حوزه‌ای `(domain partitioning)` را به منتهای خود می‌رساند؛ از بسیاری جهات، این معماری تجسم فیزیکیِ مفاهیم منطقیِ طراحی حوزه‌محور است.

دانه‌بندی

معمارانی که میکروسرویس طراحی می‌کنند اغلب برای یافتن سطح درستِ دانه‌بندی دست‌وپنجه نرم می‌کنند و در نهایت، با برداشتِ تحت‌اللفظی از واژهٔ «میکرو»، سرویس‌هایشان را بیش از حد کوچک می‌سازند. سپس ناچار می‌شوند برای انجام کاری سودمند، پیوندهای ارتباطی میان سرویس‌ها را دوباره برقرار کنند؛ کاری که هدف اصلی را بی‌اثر می‌کند و به «توپِ بزرگِ گِلِ توزیع‌شده» `(Big Ball of Distributed Mud)` می‌انجامد.

> **واژهٔ میکروسرویس یک برچسب است، نه یک توصیف.**
>
> — Martin Fowler

به بیان دیگر، پدیدآورندگان این اصطلاح ناچار بودند این سبک تازه را **به نامی** بخوانند و میکروسرویس‌ها را برگزیدند تا آن را در برابر سبک معماریِ غالبِ آن زمان (حدود سال ۲۰۰۷)، یعنی معماری سرویس‌گرا `(service-oriented architecture)`، قرار دهند؛ معماری‌ای که می‌شد آن را «سرویس‌های غول‌آسا» نامید. با این حال، بسیاری از توسعه‌دهندگان واژهٔ میکروسرویس‌ها را یک فرمان تلقی می‌کنند، نه یک توصیف، و سرویس‌هایی بیش از حد ریزدانه `(fine-grained)` می‌سازند.

هدف از مرزهای سرویس در میکروسرویس‌ها، دربرگرفتنِ یک حوزه یا گردش‌کار است. در برخی برنامه‌های کاربردی، این مرزهای طبیعی ممکن است برای بخش‌هایی از سیستم بزرگ باشند، صرفاً به این دلیل که برخی فرایندهای کسب‌وکار کوپلینگ بیشتری با هم دارند تا دیگر فرایندها. در ادامه چند رهنمود آمده است که معماران می‌توانند برای یافتن مرزهای مناسب از آن‌ها کمک بگیرند:

هدف **`(Purpose)`**: آشکارترین مرز بر منبع الهام این سبک معماری تکیه دارد: حوزهٔ مسئله. در حالت آرمانی، هر میکروسرویس باید انسجام کارکردی `(functional cohesion)` داشته باشد و یک رفتار مهم را از جانب کل برنامهٔ کاربردی بر عهده بگیرد.

تراکنش‌ها **`(Transactions)`**: زمینه‌های محدود گردش‌کارهای کسب‌وکارند و اغلب موجودیت‌هایی `(entities)` که باید در یک تراکنش با هم همکاری کنند، مرز مناسبی برای سرویس پیشنهاد می‌دهند. از آنجا که تراکنش‌ها در معماری‌های توزیع‌شده مشکل‌ساز هستند، طراحیِ سیستم‌ها با هدف پرهیز از آن‌ها معمولاً به طراحی‌های بهتری می‌انجامد.

کرئوگرافی **`(Choreography)`**: گاهی مجموعه‌ای از سرویس‌ها جداسازیِ حوزه‌ایِ عالی‌ای فراهم می‌کنند، اما برای کار کردن به ارتباطات گسترده نیاز دارند. در این حالت، معمار می‌تواند ادغام دوبارهٔ این سرویس‌ها در یک سرویس بزرگ‌تر را بررسی کند تا از سربار ارتباطی پرهیز شود.

تکرار `(iteration)` تنها راه تضمین طراحی خوب سرویس‌هاست. معماران به‌ندرت سطح کاملِ دانه‌بندی، وابستگی‌های داده و سبک‌های ارتباطی را در نخستین دور کشف می‌کنند؛ آن‌ها گزینه‌ها را بارها مرور می‌کنند تا طراحی‌شان را پالایش کنند، به‌ویژه هرچه بیشتر دربارهٔ سیستم و کارکردهای کسب‌وکارِ آن می‌آموزند.

جداسازی داده

یکی دیگر از الزامات میکروسرویس‌ها، که آن هم از مفهوم زمینهٔ محدود سرچشمه می‌گیرد، **جداسازی داده** `(data isolation)` است. بسیاری از سبک‌های معماری برای ماندگاری داده‌ها از یک پایگاه‌دادهٔ واحد استفاده می‌کنند. اما سبک میکروسرویس‌ها می‌کوشد از **همهٔ** انواع کوپلینگ پرهیز کند؛ از جمله استفاده از طرح‌واره‌ها و پایگاه‌داده‌های مشترک به‌عنوان نقطه‌های یکپارچه‌سازی.

جداسازی داده عامل دیگری است که هنگام بررسی دانه‌بندی سرویس‌ها باید در نظر گرفت. از تلهٔ موجودیت (Entity Trap؛ که در بخش «تلهٔ موجودیت» در فصل ۸ بررسی شد) برحذر باشید: سرویس‌ها را صرفاً به شکل موجودیت‌های منفرد یک پایگاه‌داده مدل نکنید. معماران عادت دارند از پایگاه‌داده‌های رابطه‌ای `(relational databases)` برای یکپارچه‌کردن مقادیر درون سیستم استفاده کنند و به این ترتیب یک منبع واحد حقیقت `(single source of truth)` بسازند؛ اما وقتی داده‌ها را در سراسر معماری توزیع می‌کنید، این دیگر گزینه‌ای در دسترس نیست. بنابراین هر معمار باید تصمیم بگیرد که می‌خواهد این مسئله را چگونه حل کند: یا یک حوزه را منبع حقیقتِ یک واقعیت مشخص تعیین کند و برای بازیابی مقادیر با آن هماهنگ شود، یا اطلاعات را از راه تکثیر پایگاه‌داده `(database replication)` یا کش‌گذاری `(caching)` توزیع کند.

این سطح از جداسازی داده دردسرهایی پدید می‌آورد، اما فرصت‌هایی هم فراهم می‌کند. اکنون که تیم‌ها ناچار نیستند حول یک پایگاه‌دادهٔ واحد یکپارچه شوند، هر تیم می‌تواند مناسب‌ترین فناوری پایگاه‌داده را بر پایهٔ بودجهٔ سرویس خود، نوع ساختار ذخیره‌سازی، ویژگی‌های عملیاتی، ویژگی‌های فرایندی و مانند آن برگزینند. مزیت دیگر یک سیستم با کوپلینگ

بسیار کم این است که هر تیم می‌تواند مسیرش را تغییر دهد و پایگاه‌داده (یا وابستگی دیگری) مناسب‌تر برگزیند، بی‌آنکه بر تیم‌های دیگر اثر بگذارد؛ زیرا آن تیم‌ها اجازه ندارند به جزئیات پیاده‌سازی کوپل شوند. (جداسازی داده و ملاحظات پایگاه‌داده را در بخش «توپولوژی‌های داده» در همین فصل با جزئیات بیشتری بررسی می‌کنیم.)

لایهٔ `API`

بیشتر معماری‌های میکروسرویس یک لایهٔ `API` `(API layer)` دارند که معمولاً **دروازهٔ** **`API`** `(API Gateway)` نامیده می‌شود و میان مصرف‌کنندگان سیستم (چه رابط‌های کاربری و چه فراخوانی‌هایی از سیستم‌های دیگر) و میکروسرویس‌ها قرار می‌گیرد. لایهٔ `API` را می‌توان به شکل یک پراکسی معکوس `(reverse proxy)` ساده پیاده‌سازی کرد، یا به شکل دروازه‌ای پیچیده‌تر که دغدغه‌های میان‌بُرشی `(cross-cutting concerns)` مانند امنیت، سرویس‌های نام‌گذاری `(naming services)` و مانند آن را در خود دارد (این موارد را در بخش «بازاستفادهٔ عملیاتی» با جزئیات بیشتری بررسی می‌کنیم).

لایه‌های `API` کاربردهای فراوانی دارند، اما برای وفادار ماندن به فلسفهٔ زیربنایی این معماری، نباید از آن‌ها به‌عنوان میانجی `(mediator)` یا هماهنگ‌کننده `(orchestrator)` استفاده کرد. همهٔ منطق کسب‌وکاریِ مهم در این معماری باید درون یک زمینهٔ محدود جای گیرد و گذاشتنِ ارکستراسیون یا هر منطق کسب‌وکاری دیگری در یک میانجی این قاعده را نقض می‌کند. معماران معمولاً میانجی‌ها را در معماری‌هایی به کار می‌برند که افراز فنی `(technical partitioning)` دارند، در حالی که سبک میکروسرویس‌ها قاطعانه بر افراز حوزه‌ای استوار است.

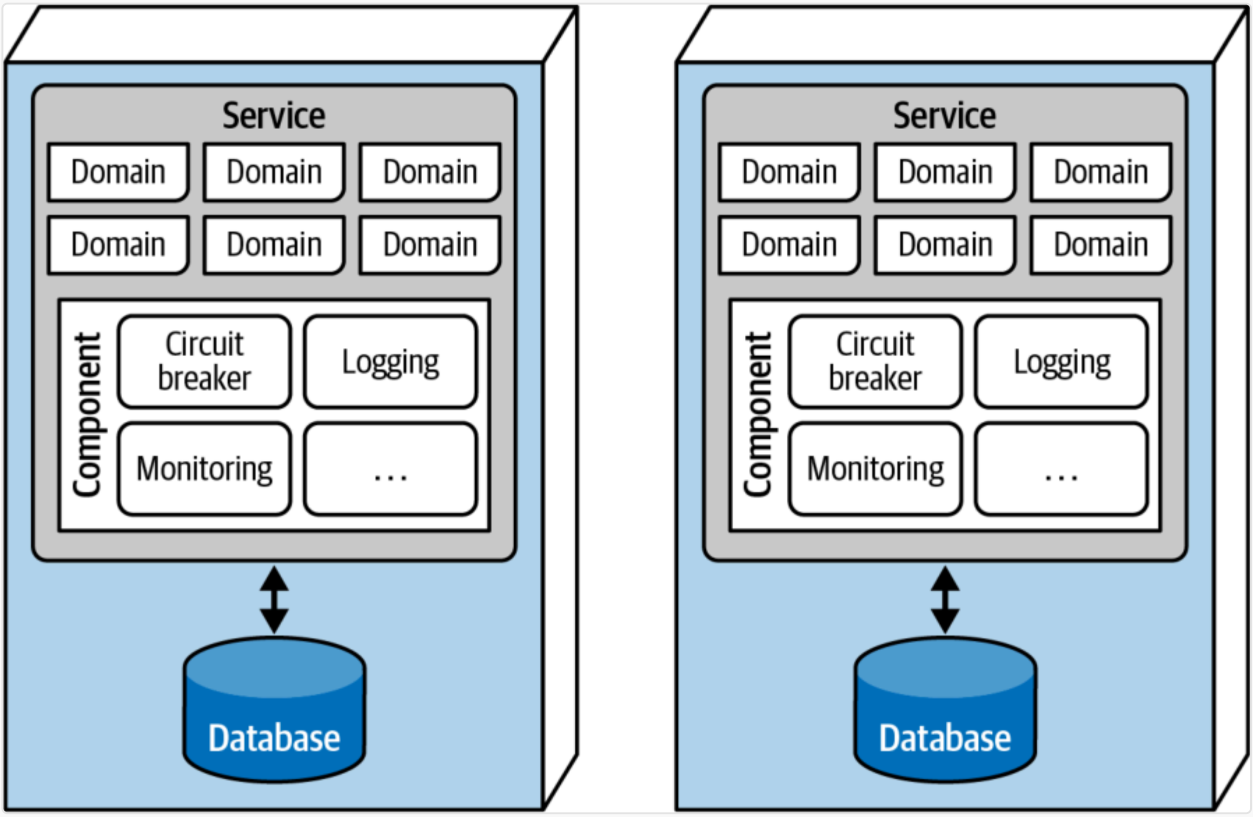
> **نکته:** هنگام استفاده از لایهٔ `API` در یک معماری میکروسرویس، فقط مسیریابی درخواست‌ها `(request routing)` و دغدغه‌های میان‌بُرشی مانند امنیت، پایش `(monitoring)`، ثبت وقایع `(logging)` و مانند آن را در آن بگنجانید. مراقب باشید هیچ منطق مرتبط با کسب‌وکار را در لایهٔ `API` قرار ندهید.

بازاستفادهٔ عملیاتی

با توجه به اینکه سبک میکروسرویس‌ها دوباره‌نویسی را بر کوپلینگ ترجیح می‌دهد، معماران با بخش‌هایی از این معماری که واقعاً از کوپلینگ سود می‌برند چه می‌کنند؟ منظور دغدغه‌های عملیاتی‌ای مانند پایش، ثبت وقایع و قطع‌کننده‌های مدار `(circuit breakers)` است. فلسفهٔ سنتیِ معماری سرویس‌گرا این بود که تا حد امکان کارکردها را بازاستفاده کند؛ چه کارکردهای حوزه‌ای و چه کارکردهای عملیاتی. اما در میکروسرویس‌ها، معماران می‌کوشند این دو دغدغه را از هم جدا کنند.

وقتی تیمی چندین میکروسرویس ساخت، اعضایش کم‌کم درمی‌یابند که هر میکروسرویس عناصر مشترکی دارد که از یکسانی سود می‌برند. برای مثال، اگر سازمانی به هر تیم سرویس اجازه دهد پایش را مستقلاً پیاده‌سازی کند، چگونه می‌تواند مطمئن شود که هر تیم واقعاً این کار را انجام می‌دهد؟ و سازمان با دغدغه‌هایی مانند ارتقا چگونه کنار می‌آید؟

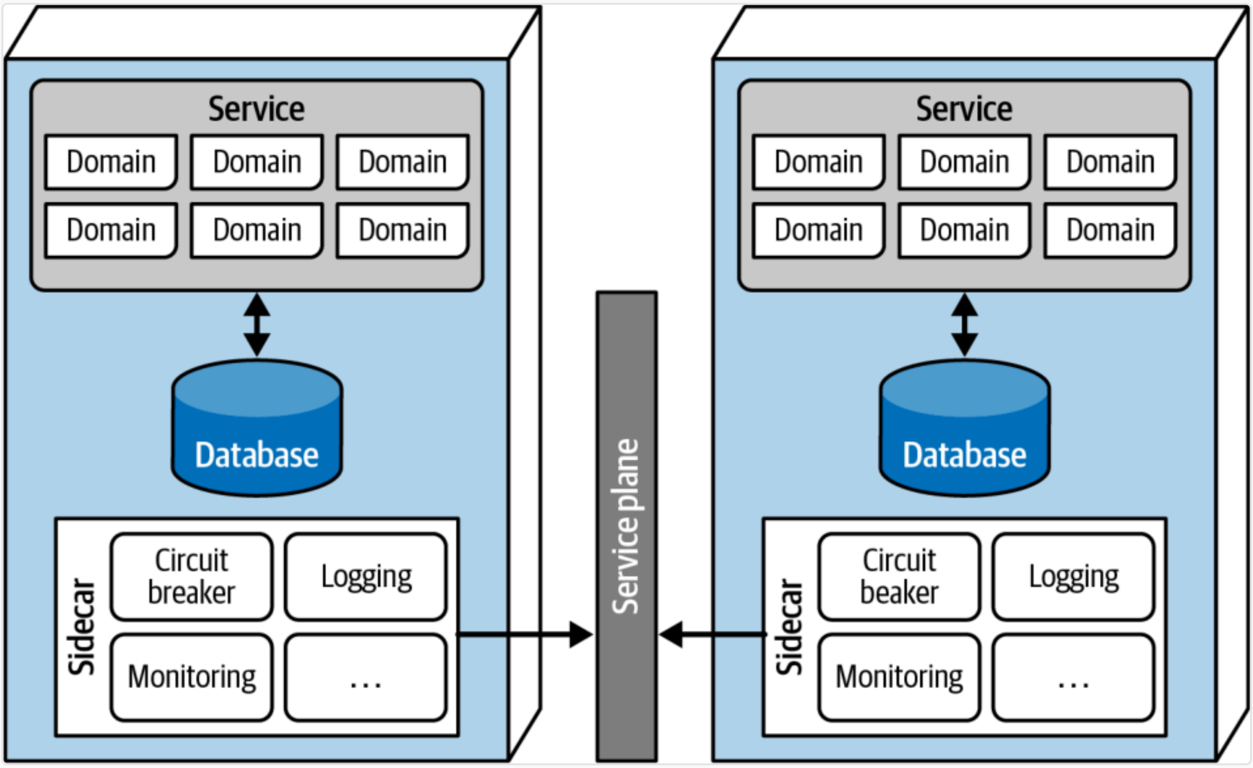
آیا هر تیم مسئول ارتقا به نسخهٔ جدید ابزار پایش می‌شود، و این کار چقدر طول خواهد کشید؟ الگوی **`Sidecar`** راه‌حلی برای این مسئله ارائه می‌کند (شکل ۱۸-۲).


Service
Domain
Domain
Domain
Domain
Domain
Domain
Component
Circuit breaker
Logging
Monitoring
...
Database
Service
Domain
Domain
Domain
Domain
Domain
Domain
Component
Circuit breaker
Logging
Monitoring
...
Database

شکل ۱۸-۲. الگوی `Sidecar` در میکروسرویس‌ها

در شکل ۱۸-۲، دغدغه‌های عملیاتیِ مشترک (قطع‌کنندهٔ مدار، ثبت وقایع، پایش) درون هر سرویس به شکل مؤلفه‌هایی جداگانه ظاهر می‌شوند که مالکیت هر کدام می‌تواند بر عهدهٔ تیم‌های مستقل یا یک تیم زیرساختِ مشترک باشد. مؤلفهٔ `Sidecar` همهٔ دغدغه‌های عملیاتی‌ای را که از کوپلینگ سود می‌برند بر عهده می‌گیرد؛ بنابراین وقتی زمان ارتقای ابزار پایش فرا می‌رسد، تیم زیرساختِ مشترک می‌تواند `sidecar` را به‌روزرسانی کند و هر میکروسرویس کارکرد جدید را دریافت می‌کند (نگاه کنید به بخش «ملاحظاتِ توپولوژی تیم» در همین فصل).

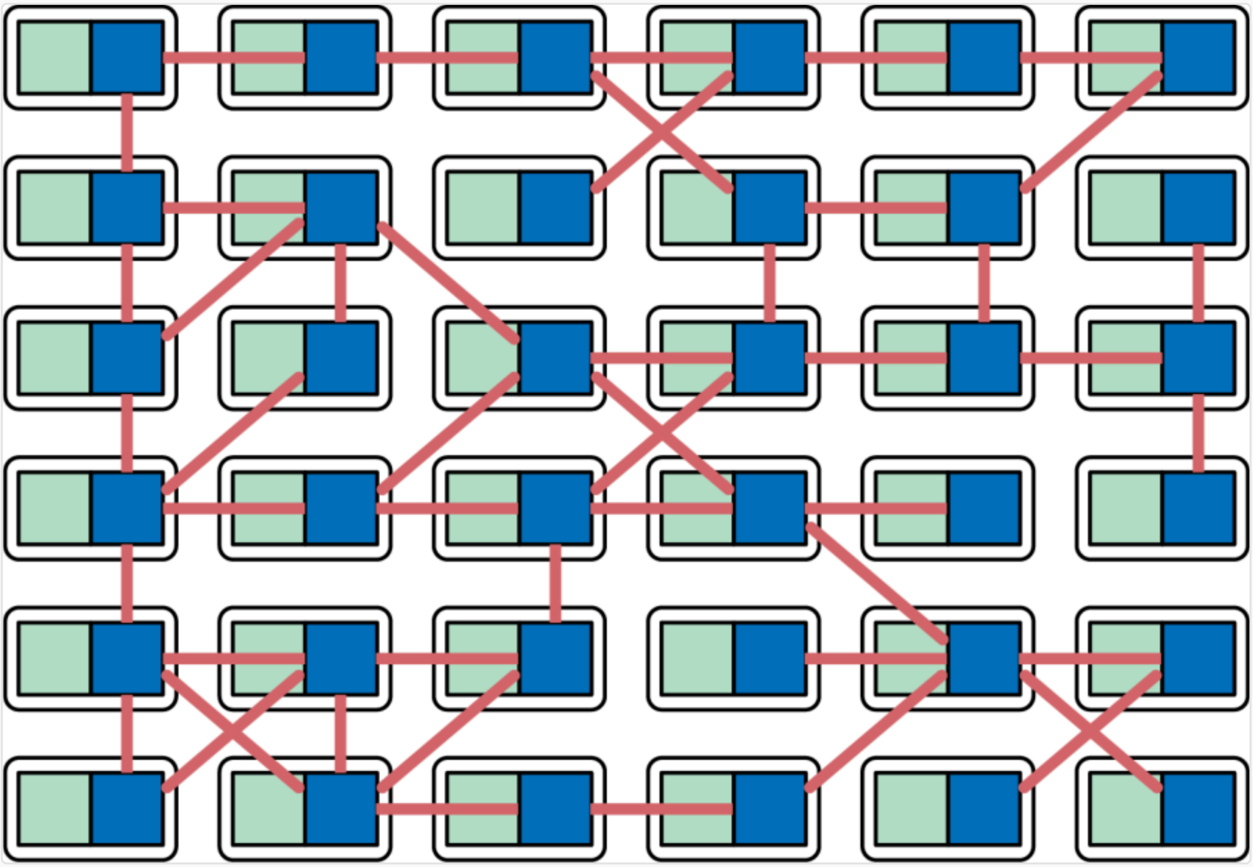
وقتی هر سرویس یک مؤلفهٔ `Sidecar` مشترک داشته باشد، معمار می‌تواند یک **مش سرویس** `(service mesh)` بسازد تا تیم‌ها کنترلی یکپارچه بر این دغدغه‌های مشترک در سراسر معماری داشته باشند. مؤلفه‌های `sidecar` به هم متصل می‌شوند و یک واسط عملیاتیِ یکدست در سراسر همهٔ میکروسرویس‌ها پدید می‌آورند؛ چنان‌که در شکل ۱۸-۳ نشان داده شده است.


Service
Domain
Domain
Domain
Domain
Domain
Domain
Database
Sidecar
Circuit breaker
Logging
Monitoring
...
Service plane
Service
Domain
Domain
Domain
Domain
Domain
Domain
Database
Sidecar
Circuit beaker
Logging
Monitoring
...

شکل ۱۸-۳. صفحهٔ سرویس، `sidecar`ها را در یک مش سرویس به هم متصل می‌کند

در شکل ۱۸-۳، هر `sidecar` به **صفحهٔ سرویس** `(service plane)` وصل می‌شود. صفحهٔ سرویس نرم‌افزاری یکپارچه‌ساز است (معمولاً در قالب محصولی مانند `Istio`) که هر `sidecar` را از راه یک واسط یکدست متصل می‌کند و به این ترتیب مش سرویس را شکل می‌دهد.

هر سرویس یک گره `(node)` در کل مش را تشکیل می‌دهد؛ چنان‌که در شکل ۱۸-۴ نشان داده شده است. مش سرویس کنسولی پدید می‌آورد که به تیم‌ها امکان می‌دهد کوپلینگ عملیاتی، مانند سطوح پایش، ثبت وقایع و دیگر دغدغه‌های عملیاتیِ میان‌بُرشی، را به‌صورت سراسری کنترل کنند.



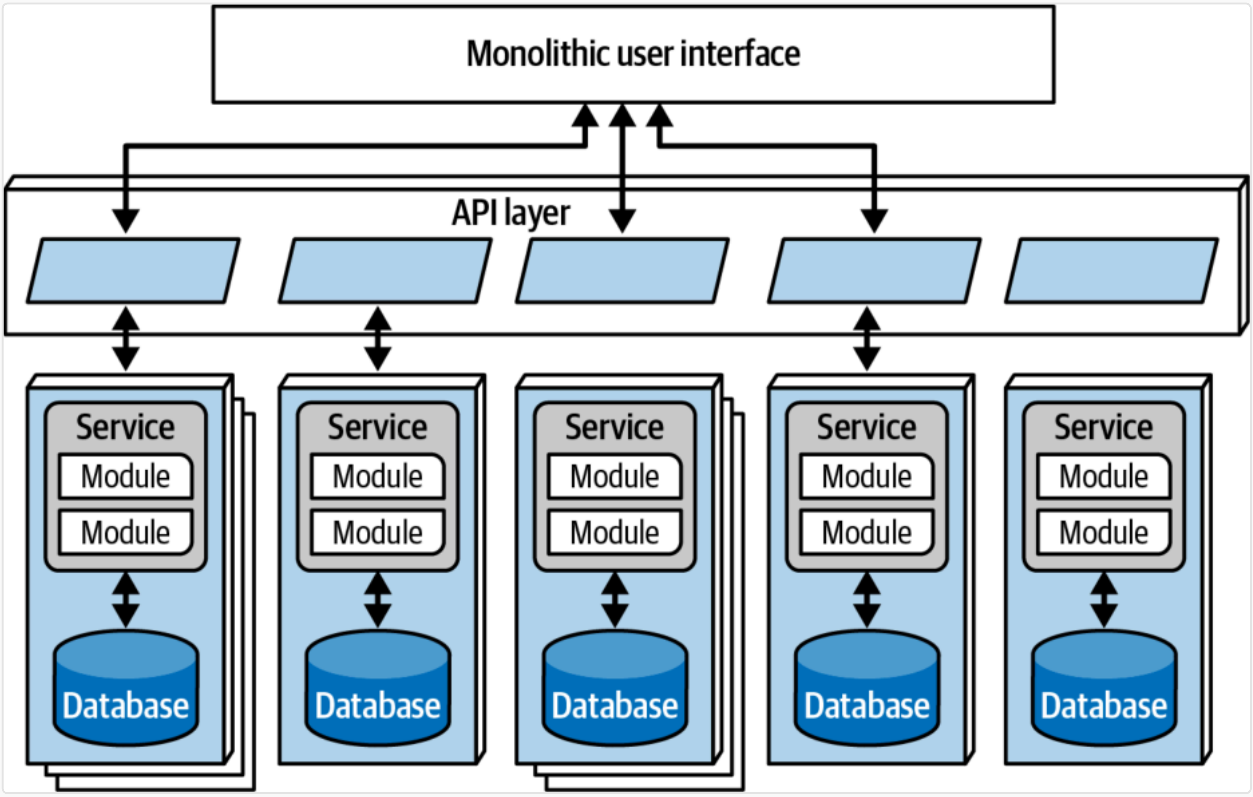
شکل ۱۸-۴. مش سرویس نمایی کل‌نگر از جنبهٔ عملیاتیِ میکروسرویس‌ها پدید می‌آورد

معماران از کشف سرویس `(service discovery)` برای گنجاندنِ کشسانی `(elasticity)` در معماری‌های میکروسرویس استفاده می‌کنند. **کشف سرویس** روشی برای شناسایی و مکان‌یابیِ خودکارِ سرویس‌ها درون یک شبکه است. وقتی درخواستی می‌رسد، به جای آنکه یک سرویس واحد را فراخوانی کند، از ابزار کشف سرویس عبور می‌کند؛ ابزاری که می‌تواند تعداد و بسامد درخواست‌ها را پایش کند و برای پاسخ به دغدغه‌های مقیاس‌پذیری یا کشسانی، نمونه‌های `(instances)` جدیدی از سرویس‌ها راه‌اندازی کند. معماران اغلب کشف سرویس را در مش سرویس می‌گنجانند و آن را بخشی از هر میکروسرویس می‌کنند. لایهٔ `API` نیز اغلب میزبان کشف سرویس است و جایگاهی واحد فراهم می‌کند تا رابط‌های کاربری یا دیگر سیستم‌های فراخواننده، سرویس‌ها را به شیوه‌ای کشسان و یکدست بیابند و ایجاد کنند.

سمت کاربر

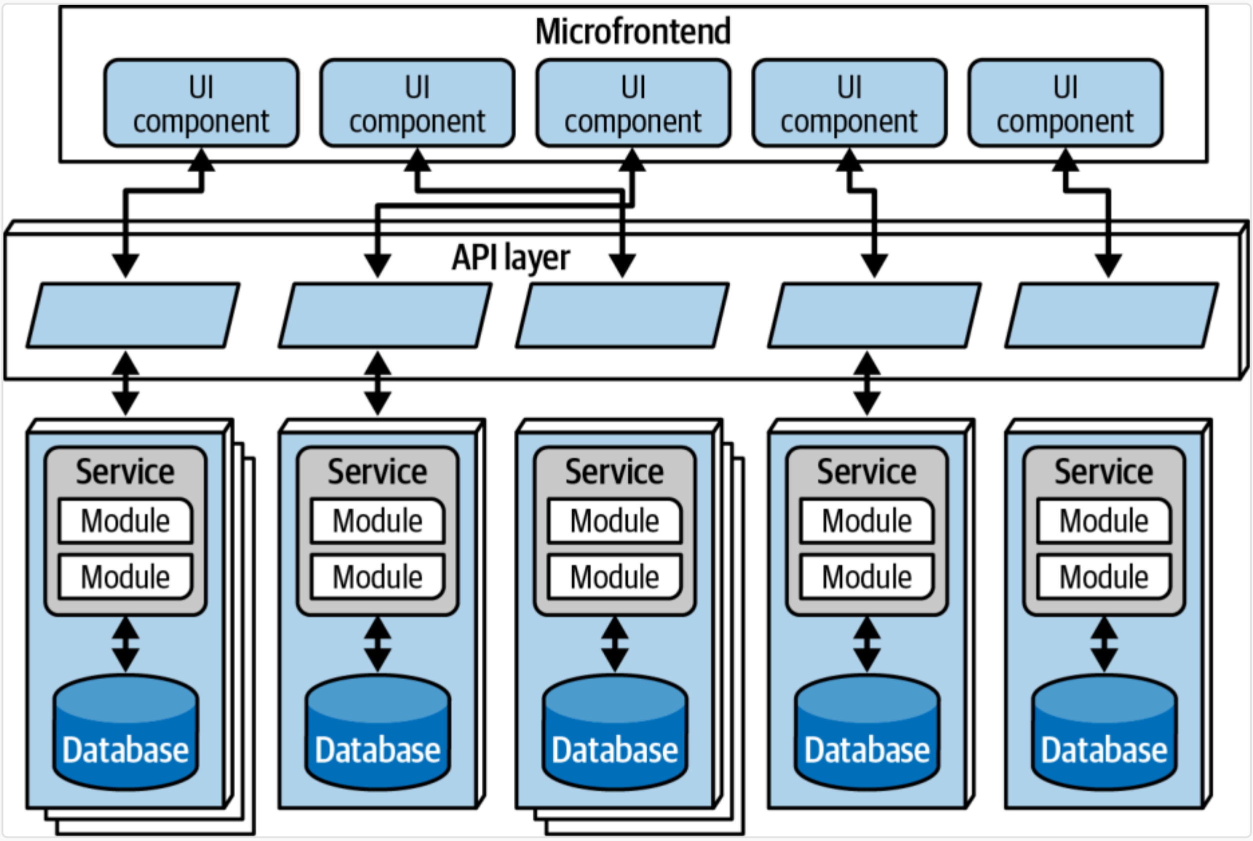
سبک میکروسرویس‌ها جداسازی را ترجیح می‌دهد و این جداسازی در حالت آرمانی باید رابط‌های کاربری را هم، در کنار دغدغه‌های سمت سرور، در بر بگیرد. در واقع، تصویر اولیهٔ میکروسرویس‌ها، به پیروی از اصل زمینهٔ محدود در `DDD`، رابط کاربری `(UI)` را بخشی از زمینهٔ محدود می‌دانست. اما ملاحظات عملیِ افرازی که برنامه‌های کاربردی وب (و دیگر قیدهای بیرونی) ایجاب می‌کنند، دستیابی به این هدف را دشوار می‌کند. به همین دلیل، معمولاً دو سبک رابط کاربری را برای معماری‌های میکروسرویس می‌بینیم.

سبک نخست، که در شکل ۱۸-۵ نشان داده شده، **سمت کاربرِ تک‌پارچه** `(monolithic frontend)` است: یک رابط کاربری واحد که برای برآوردن درخواست‌های کاربر، از طریق لایهٔ `API` فراخوانی انجام می‌دهد. این سمت کاربر می‌تواند یک برنامهٔ دسکتاپ غنی، برنامهٔ موبایل یا برنامهٔ وب باشد. برای مثال، امروزه بسیاری از برنامه‌های وب برای ساختن یک رابط کاربری واحد از یک چارچوب وب `JavaScript` استفاده می‌کنند.


Monolithic user interface
API layer
Service
Module
Module
Database
Service
Module
Module
Database
Service
Module
Module
Database
Service
Module
Module
Database
Service
Module
Module
Database

شکل ۱۸-۵. معماری میکروسرویس‌ها با رابط کاربری تک‌پارچه

گزینهٔ دوم برای رابط کاربری، **میکروفرانت‌اندها** `(micro-frontends)` است که در شکل ۱۸-۶ نشان داده شده است.


Microfrontend
UI component
UI component
UI component
UI component
UI component
API layer
Service
Module
Module
Database

شکل ۱۸-۶. الگوی میکروفرانت‌اند در میکروسرویس‌ها

رویکرد میکروفرانت‌اند از مؤلفه‌هایی در سطح رابط کاربری استفاده می‌کند تا میان رابط کاربری و سرویس‌های سمت سرور، سطحی هم‌گام از دانه‌بندی و جداسازی پدید آورد و به این ترتیب میان مؤلفه‌های رابط کاربری و سرویس‌های سمت سرورِ متناظرشان پیوند برقرار کند.

برای آشنایی بیشتر با میکروفرانت‌اندها، کتابِ **Building Micro-Frontends**، ویراست دوم، نوشتهٔ Luca Mezzalira (O’Reilly، ۲۰۲۵) را قویاً توصیه می‌کنیم.

ارتباطات

در میکروسرویس‌ها، معماران و توسعه‌دهندگان برای یافتن دانه‌بندی مناسب سرویس‌ها دست‌وپنجه نرم می‌کنند؛ دانه‌بندی‌ای که هم بر جداسازی داده اثر می‌گذارد و هم بر ارتباطات. یافتن سبک ارتباطی درست به تیم‌ها کمک می‌کند سرویس‌ها را جدا از هم نگه دارند و در عین حال آن‌ها را به شیوه‌هایی سودمند هماهنگ کنند.

در اصل، معماران باید میان ارتباط **هم‌گام** `(synchronous)` و **ناهم‌گام** `(asynchronous)` تصمیم بگیرند. در ارتباط هم‌گام، فرستنده باید منتظر پاسخ گیرنده بماند. معماری‌های میکروسرویس معمولاً در ارتباط با سرویس‌ها و میان آن‌ها از **همکنش‌پذیریِ ناهمگونِ آگاه از پروتکل** `(protocol-aware heterogeneous interoperability)` بهره می‌گیرند. بیایید این اصطلاح پیچیده را به اجزایش بشکنیم تا بهتر بفهمیم چه معنایی دارد و چرا اهمیت دارد:

آگاه از پروتکل `(Protocol-aware)`: از آنجا که میکروسرویس‌ها هاب یکپارچه‌سازیِ `(integration hub)` متمرکز ندارند، هر سرویس باید بداند سرویس‌های دیگر را **چگونه** فراخوانی کند. به همین دلیل، معماران معمولاً شیوهٔ فراخوانیِ سرویس‌های مشخص از سوی یکدیگر را استاندارد می‌کنند: سطح مشخصی از `REST`، صف‌های پیام `(message queues)` و مانند آن. این یعنی سرویس‌ها باید بدانند (یا کشف کنند) که برای فراخوانی سرویس‌های دیگر از کدام پروتکل استفاده کنند.

ناهمگون `(Heterogeneous)`: از آنجا که میکروسرویس‌ها یک معماری توزیع‌شده است، هر سرویس می‌تواند با پشتهٔ فناوری `(technology stack)` متفاوتی نوشته شود. «ناهمگون» نشان می‌دهد که سبک میکروسرویس‌ها از محیط‌های چندزبانه `(polyglot environments)`، یعنی محیط‌هایی که در آن‌ها سرویس‌های مختلف از پلتفرم‌های مختلف استفاده می‌کنند، به‌طور کامل پشتیبانی می‌کند.

همکنش‌پذیری `(Interoperability)`: به فراخوانیِ سرویس‌ها از سوی یکدیگر اشاره دارد. هرچند معماران در میکروسرویس‌ها می‌کوشند از فراخوانی‌های متدِ تراکنشی جلوگیری کنند، سرویس‌ها معمولاً برای همکاری و تبادل اطلاعات، سرویس‌های دیگر را از طریق شبکه فراخوانی می‌کنند.

ناهمگونیِ اجباری `(Enforced Heterogeneity)`

یکی از معماران سرشناس و پیشگامان سبک میکروسرویس‌ها، معمار ارشد یک استارتاپ بود که نرم‌افزار مدیریت اطلاعات شخصی برای دستگاه‌های همراه می‌ساخت. از آنجا که موبایل حوزهٔ مسئله‌ای است که بسیار سریع تغییر می‌کند، این معمار می‌خواست مطمئن شود هیچ‌یک از تیم‌های توسعه به‌طور ناخواسته نقطه‌های کوپلینگی پدید نمی‌آورد که توانایی آن‌ها را در حرکت مستقل محدود کند. معلوم شد که مهارت‌های فنی تیم‌ها ترکیب بسیار متنوعی دارد؛ از این رو، معمار قاعدهٔ تازه‌ای وضع کرد: هر تیم توسعه باید از پشتهٔ فناوریِ **متفاوتی** استفاده کند. اگر یک تیم از `Java` و تیم دیگر از `.NET` استفاده می‌کرد، به اشتراک‌گذاشتنِ ناخواستهٔ کلاس‌ها میان آن‌ها ناممکن می‌شد!

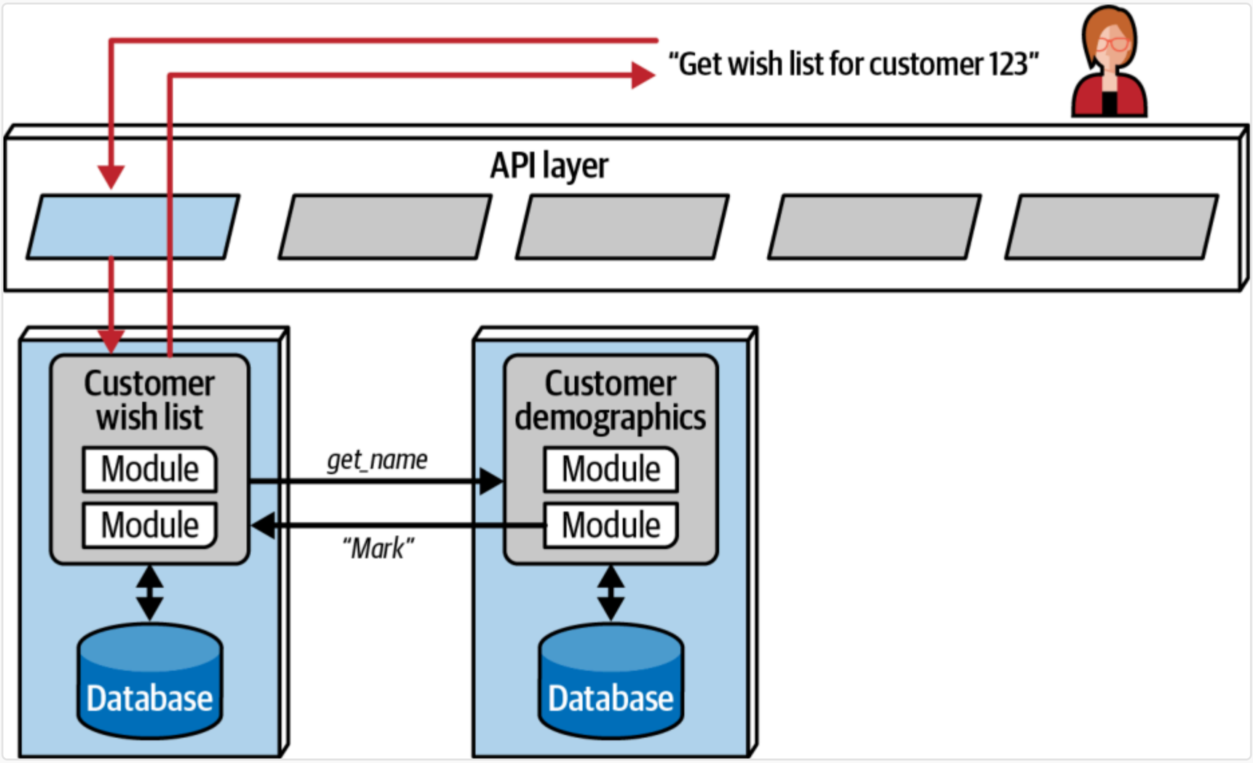
این رویکرد درست نقطهٔ مقابل بیشتر سیاست‌های حاکمیتی سازمانی است که بر استانداردسازی روی یک پشتهٔ فناوریِ واحد پافشاری می‌کنند. در دنیای میکروسرویس‌ها، هدف ساختنِ پیچیده‌ترین اکوسیستم ممکن نیست، بلکه برگزیدنِ مقیاس درستِ فناوری برای دامنهٔ محدودِ مسئله است. هر سرویسی به یک پایگاه‌دادهٔ رابطه‌ایِ صنعتی و قدرتمند نیاز ندارد و تحمیل چنین پایگاه‌داده‌ای به یک تیم کوچک به احتمال زیاد بیشتر سرعتشان را می‌گیرد تا اینکه سودی برایشان داشته باشد. این مفهوم از ماهیتِ به‌شدت جداشدهٔ میکروسرویس‌ها بهره می‌گیرد.

برای ارتباط ناهم‌گام، معماران اغلب از رویدادها و پیام‌ها استفاده می‌کنند؛ مشابه آنچه در فصل ۱۵ دربارهٔ معماری رویدادمحور شرح دادیم.

کرئوگرافی و ارکستراسیون

کرئوگرافی `(choreography)` از همان سبک ارتباطی معماری رویدادمحور `(EDA)` بهره می‌گیرد. معماری‌های کرئوگرافی‌شده هیچ هماهنگ‌کنندهٔ مرکزی ندارند؛ به این ترتیب به فلسفهٔ زمینهٔ محدود احترام می‌گذارند و پیاده‌سازیِ رویدادهای جداشده میان سرویس‌ها را طبیعی می‌کنند.

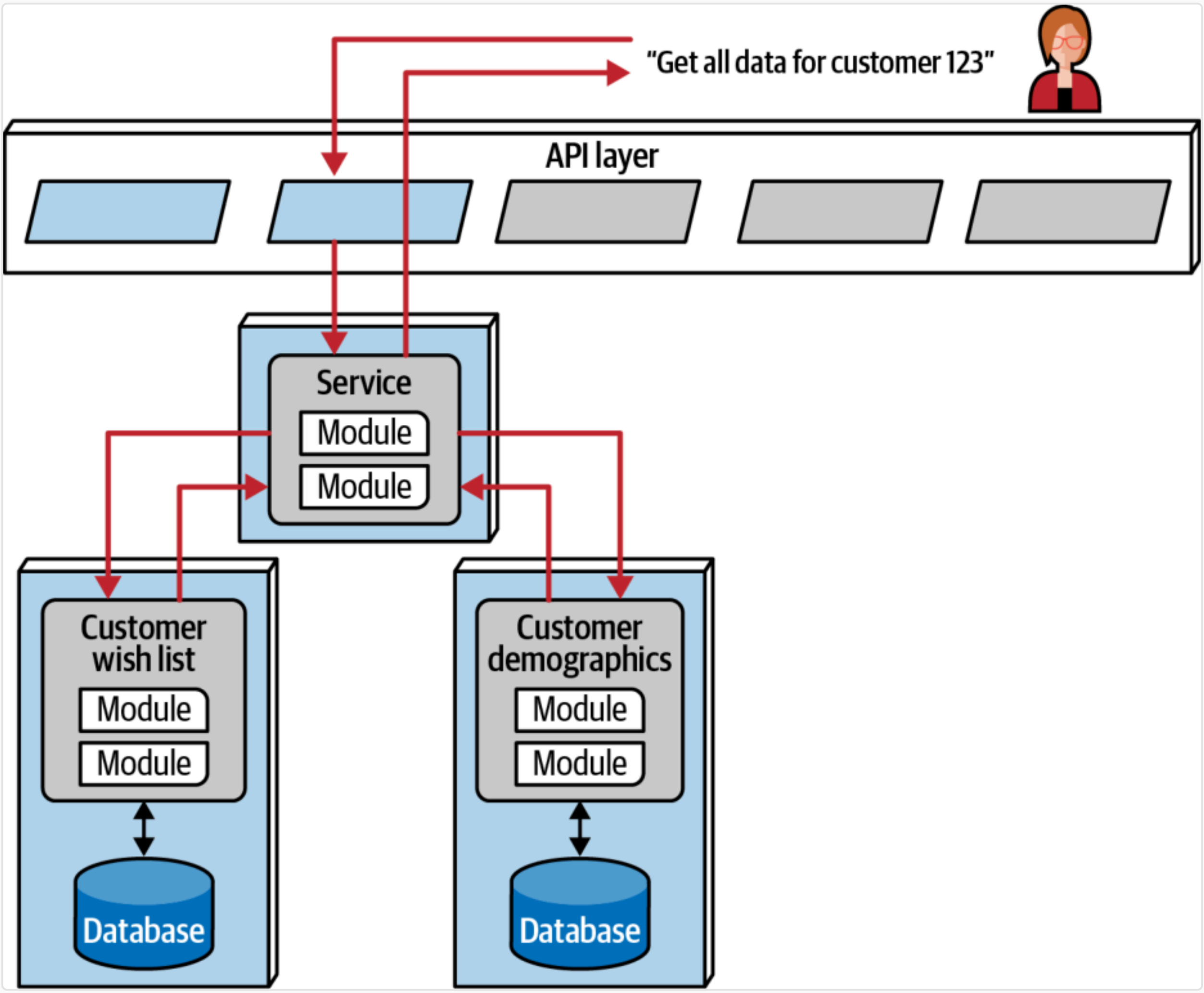
در کرئوگرافی، هر سرویس در صورت نیاز سرویس‌های دیگر را فراخوانی می‌کند، بی‌آنکه میانجیِ مرکزی در کار باشد. برای مثال، سناریوی نشان‌داده‌شده در شکل ۱۸-۷ را در نظر بگیرید. کاربر جزئیات فهرست علاقه‌مندی‌های `(wish list)` کاربر دیگری را درخواست می‌کند. از آنجا که سرویس `CustomerWishList` همهٔ اطلاعات لازم را در اختیار ندارد، سرویس `CustomerDemographics` را فراخوانی می‌کند تا اطلاعات کم‌شده را بازیابی کند و سپس نتیجه را به کاربر بازمی‌گرداند.


"Get wish list for customer 123"
API layer
Customer wish list
Module
Module
Database
get_name
"Mark"
Customer demographics
Module
Module
Database

شکل ۱۸-۷. استفاده از کرئوگرافی در میکروسرویس‌ها برای مدیریت هماهنگی

از آنجا که معماری‌های میکروسرویس، برخلاف دیگر معماری‌های سرویس‌گرا، میانجیِ سراسری ندارند، اگر معماری نیاز داشته باشد میان چند سرویس هماهنگی برقرار کند، می‌تواند میانجیِ محلیِ خودش را بسازد (که معمولاً **سرویس ارکستراسیون** `(orchestration service)` نامیده می‌شود).

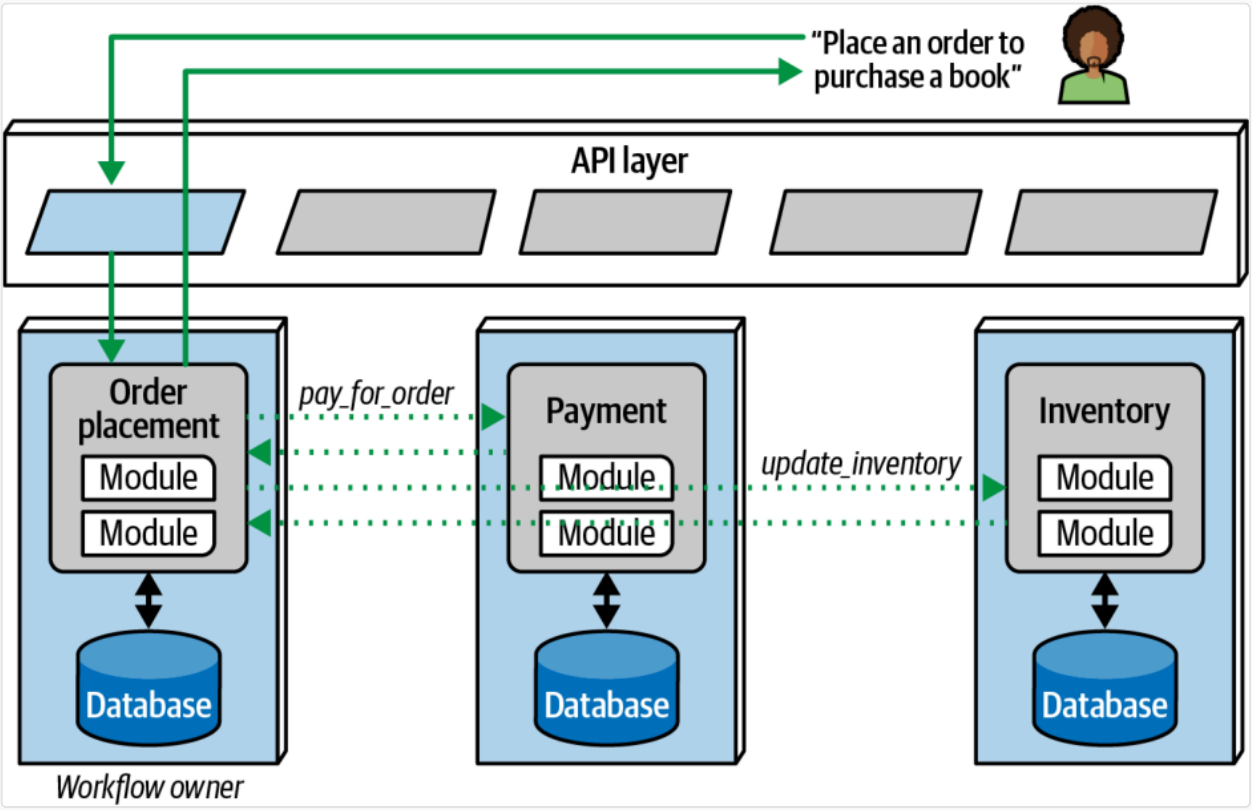
در شکل ۱۸-۸، توسعه‌دهندگان سرویسی می‌سازند که تنها مسئولیتش هماهنگ‌کردن فراخوانی‌هاست. برای نمونه، کاربر میانجیِ `ReportCustomerInformation` را فراخوانی می‌کند و این میانجی همهٔ سرویس‌های لازمِ دیگر را برای به دست آوردن اطلاعات درخواستی فرا می‌خواند.


"Get all data for customer 123"
API layer
Service
Module
Module
Customer wish list
Module
Module
Database
Customer demographics
Module
Module
Database

شکل ۱۸-۸. استفاده از ارکستراسیون در میکروسرویس‌ها

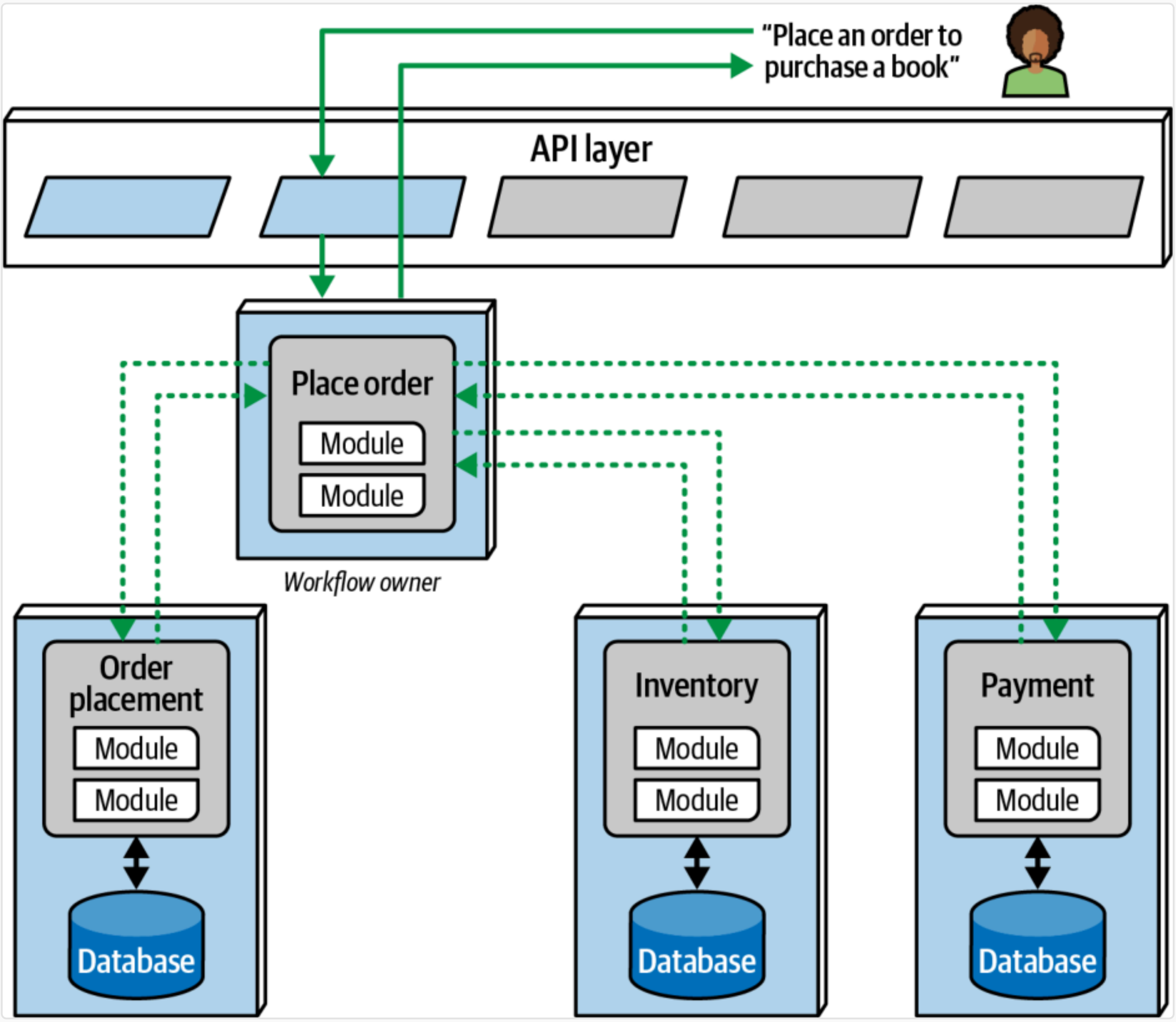
قانون اول معماری نرم‌افزار حکم می‌کند که هیچ‌یک از این دو راه‌حل بی‌نقص نیست؛ هر کدام مصالحه‌های خود را دارد. کرئوگرافی فلسفهٔ جداسازیِ حداکثریِ میکروسرویس‌ها را حفظ می‌کند تا بیشترین بهره از آن به دست آید. اما در عین حال، مسائل رایجی مانند مدیریت خطا `(error handling)` و هماهنگی را پیچیده‌تر می‌کند.

نمونه‌ای با گردش‌کار پیچیده‌تر را در نظر بگیرید. در شکل ۱۸-۹، نخستین سرویسی که فراخوانی می‌شود باید میان طیف گسترده‌ای از سرویس‌های دیگر هماهنگی برقرار کند و در واقع، افزون بر مسئولیت‌های حوزه‌ایِ دیگرش، نقش میانجی را نیز ایفا می‌کند. این الگو `Front Controller` نام دارد؛ الگویی که در آن، سرویسی که در ظاهر کرئوگرافی‌شده است برای حل مسئله‌ای مشخص به میانجی‌ای پیچیده‌تر تبدیل می‌شود. عیب این الگو آن است که سرویسی که چند نقش را بر عهده می‌گیرد، پیچیدگی را افزایش می‌دهد.


"Place an order to purchase a book"
API layer
Order placement
Module
Module
Database
Workflow owner
pay_for_order
Payment
Module
Module
Database
update_inventory
Inventory
Module
Module
Database

شکل ۱۸-۹. استفاده از کرئوگرافی برای یک فرایند کسب‌وکاری پیچیده

در مقابل، معماران می‌توانند برای فرایندهای کسب‌وکاری پیچیده از ارکستراسیون استفاده کنند؛ چنان‌که در شکل ۱۸-۱۰ نشان داده شده است. معمار یک سرویس میانجی برای هماهنگ‌کردن گردش‌کار کسب‌وکار می‌سازد؛ این کار میان این سرویس‌ها کوپلینگ پدید می‌آورد، اما به معمار هم امکان می‌دهد هماهنگی را در یک سرویس واحد متمرکز کند و سرویس‌های دیگر کمتر تحت تأثیر قرار گیرند. گردش‌کارهای حوزه اغلب ذاتاً کوپل‌شده‌اند؛ بنابراین کار معمار یافتنِ شیوه‌ای برای بازنماییِ این کوپلینگ است که به بهترین شکل از اهداف حوزه و معماری هر دو پشتیبانی کند.


"Place an order to purchase a book"
API layer
Place order
Module
Module
Workflow owner
Order placement
Module
Module
Database
Inventory
Module
Module
Database
Payment
Module
Module
Database

شکل ۱۸-۱۰. استفاده از ارکستراسیون برای یک فرایند کسب‌وکاری پیچیده

تراکنش‌ها و الگوی `Saga`

معماران در میکروسرویس‌ها در آرزوی جداسازیِ حداکثری‌اند، اما اغلب با این مسئله روبه‌رو می‌شوند که تراکنش‌ها را چگونه میان سرویس‌ها هماهنگ کنند. از آنجا که سبک میکروسرویس‌ها همان سطح از جداسازی را که در معماری ترویج می‌کند در پایگاه‌داده‌ها نیز ترویج می‌کند، اتمی‌بودن `(atomicity)` که در برنامه‌های تک‌پارچه امری پیش‌پاافتاده بود، در برنامه‌های توزیع‌شده به مسئله تبدیل می‌شود.

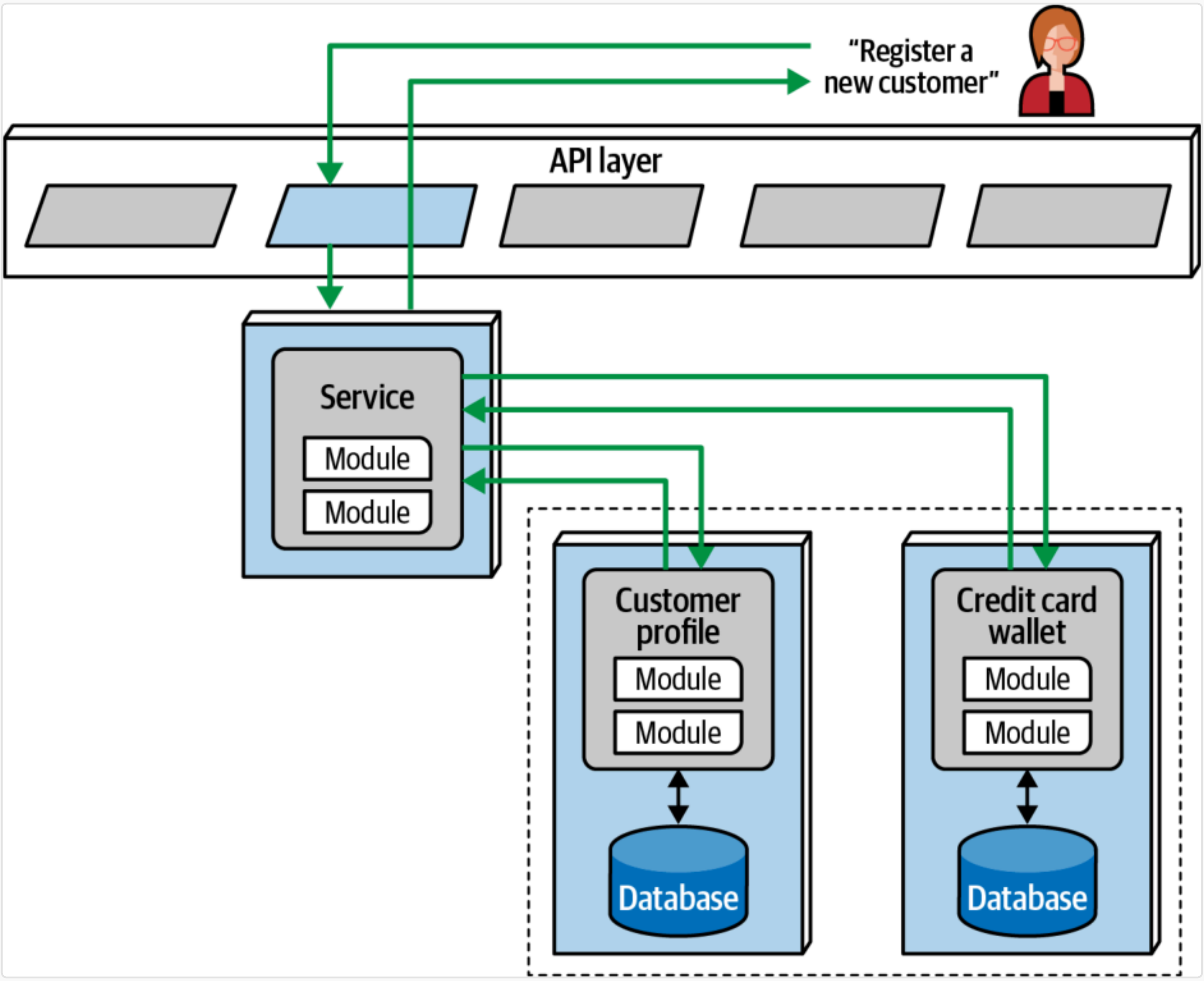
ساختنِ تراکنش‌هایی که از مرز سرویس‌ها عبور می‌کنند اصل بنیادینِ جداسازی در میکروسرویس‌ها را نقض می‌کند و افزون بر آن، بدترین نوع هم‌بستگی پویا `(dynamic connascence)`، یعنی **هم‌بستگی مقادیر** `(Connascence of Values)`، را پدید می‌آورد (نگاه کنید به بخش «هم‌بستگی» در فصل ۳). بهترین توصیه‌ای که می‌توانیم به معمارانی بکنیم که می‌خواهند میان سرویس‌ها تراکنش انجام دهند این است: **این کار را نکنید!** به جای آن، دانه‌بندی سرویس‌ها را اصلاح کنید. اگر می‌بینید که ناچارید اجزای معماری میکروسرویس خود را با تراکنش‌ها به هم وصل کنید، این نشانه‌ای است که طراحی‌تان بیش از حد ریزدانه است.

نکته: از تراکنش‌هایی که چند میکروسرویس را در بر می‌گیرند پرهیز کنید؛ به جای آن، دانه‌بندی سرویس‌ها را اصلاح کنید!

به پیروی از قاعدهٔ «بستگی دارد»، همیشه استثناهایی وجود دارد. برای مثال، ممکن است وضعیتی پیش بیاید که دو سرویس متفاوت به ویژگی‌های معماریِ بسیار متفاوتی نیاز داشته باشند که مرزهای سرویسِ جداگانه‌ای را ایجاب می‌کند، اما همچنان به هماهنگیِ تراکنشی نیاز داشته باشند. در این حالت، معمار می‌تواند پس از سنجیدنِ دقیق مصالحه‌ها، از برخی الگوهای تراکنشی برای ارکستراسیون تراکنش‌ها بهره بگیرد.

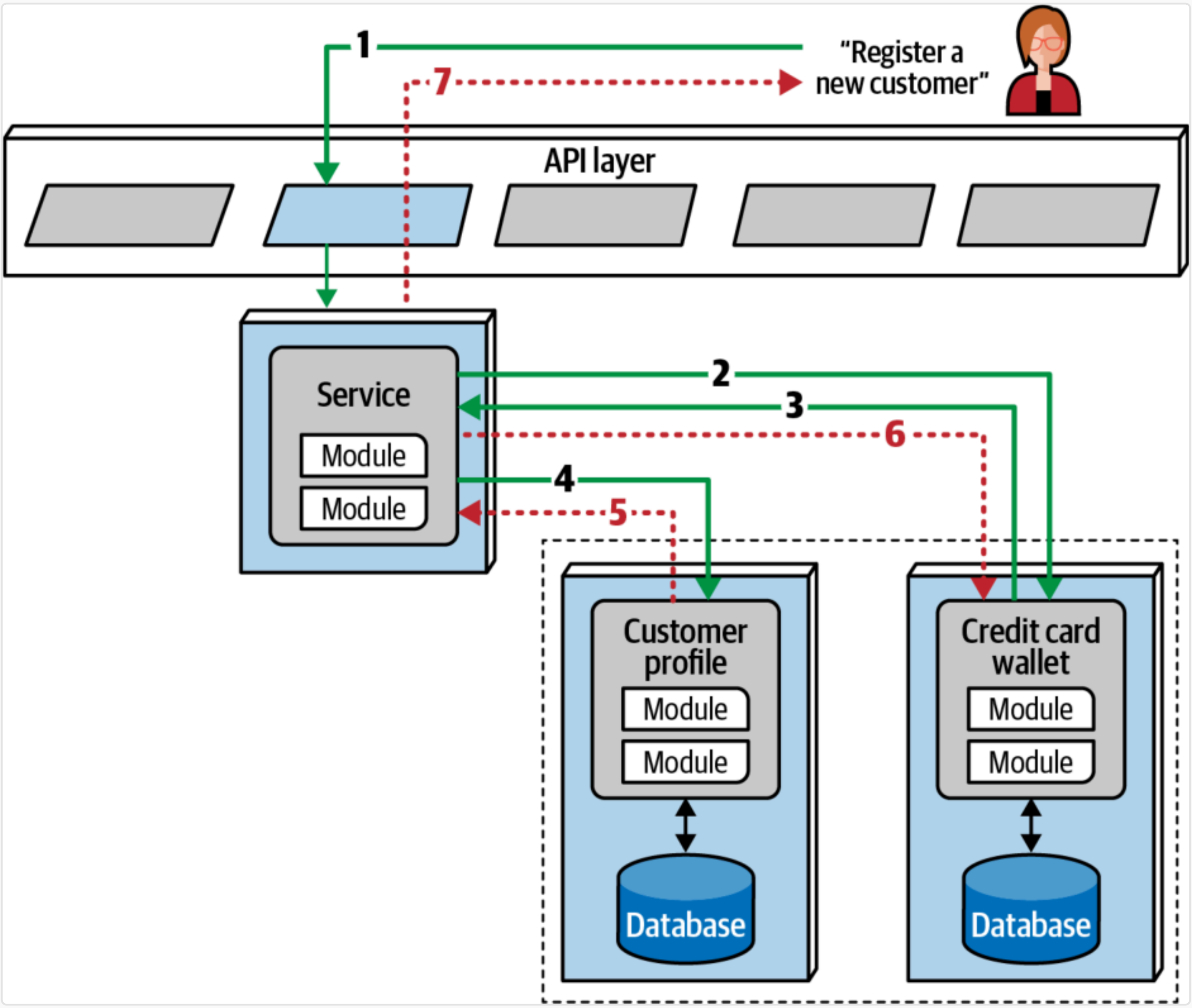
تراکنش‌های توزیع‌شده `(distributed transactions)` در میکروسرویس‌ها معمولاً با الگویی موسوم به **الگوی** **`Saga`** `(Saga pattern)` مدیریت می‌شوند. در ادبیات، `saga` (حماسه) داستانی حماسی است که زنجیرهٔ درازی از رویدادها را روایت می‌کند و به نوعی پایان قهرمانانه می‌انجامد؛ نام این الگوی تراکنشی نیز از همین‌جا آمده است.

در شکل ۱۸-۱۱، یک سرویس در نقش میانجی در سراسر چند فراخوانیِ سرویس عمل می‌کند تا یک تراکنش را هماهنگ کند. میانجی هر بخش از تراکنش را فراخوانی می‌کند، موفقیت یا شکست را ثبت می‌کند و نتایج را هماهنگ می‌سازد. اگر همه‌چیز طبق برنامه پیش برود، همهٔ مقادیر در سرویس‌ها و پایگاه‌داده‌های درون آن‌ها به‌صورت هم‌گام به‌روزرسانی می‌شوند. اگر وضعیت خطایی پیش بیاید و یک بخش از تراکنش شکست بخورد، میانجی باید مطمئن شود که هیچ بخشی از تراکنش موفق نمی‌شود. وضعیت نشان‌داده‌شده در شکل ۱۸-۱۲ را در نظر بگیرید.


"Register a new customer"
API layer
Service
Module
Module
Customer profile
Module
Module
Database
Credit card wallet
Module
Module
Database

شکل ۱۸-۱۱. الگوی `Saga` در معماری میکروسرویس‌ها

اگر بخش نخست تراکنش موفق شود اما بخش دوم شکست بخورد، میانجی باید به همهٔ سرویس‌های مشارکت‌کننده در تراکنش که موفق بوده‌اند درخواستی بفرستد و از آن‌ها بخواهد درخواست قبلی را لغو کنند. این سبک از هماهنگیِ تراکنشی **چارچوب تراکنش جبرانی** `(compensating transaction framework)` نام دارد. توسعه‌دهندگان معمولاً این الگو را به این شکل پیاده‌سازی می‌کنند که هر درخواستِ میانجی به وضعیت معلق (`pending`) وارد شود تا زمانی که میانجی موفقیت کلی را اعلام کند. با این حال، مدیریت همزمانِ درخواست‌های ناهمگام می‌تواند پیچیده شود؛ به‌ویژه اگر درخواست‌های تازه‌ای از راه برسند که به وضعیت تراکنشیِ معلق وابسته‌اند. صرف‌نظر از پروتکلی که به کار می‌رود، تراکنش‌های جبرانی ترافیک هماهنگیِ زیادی در سطح شبکه پدید می‌آورند.


1
7
"Register a new customer"
API layer
Service
Module
Module
2
3
6
4
5
Customer profile
Module
Module
Database
Credit card wallet
Module
Module
Database

شکل ۱۸-۱۲. تراکنش‌های جبرانیِ الگوی `Saga` برای وضعیت‌های خطا

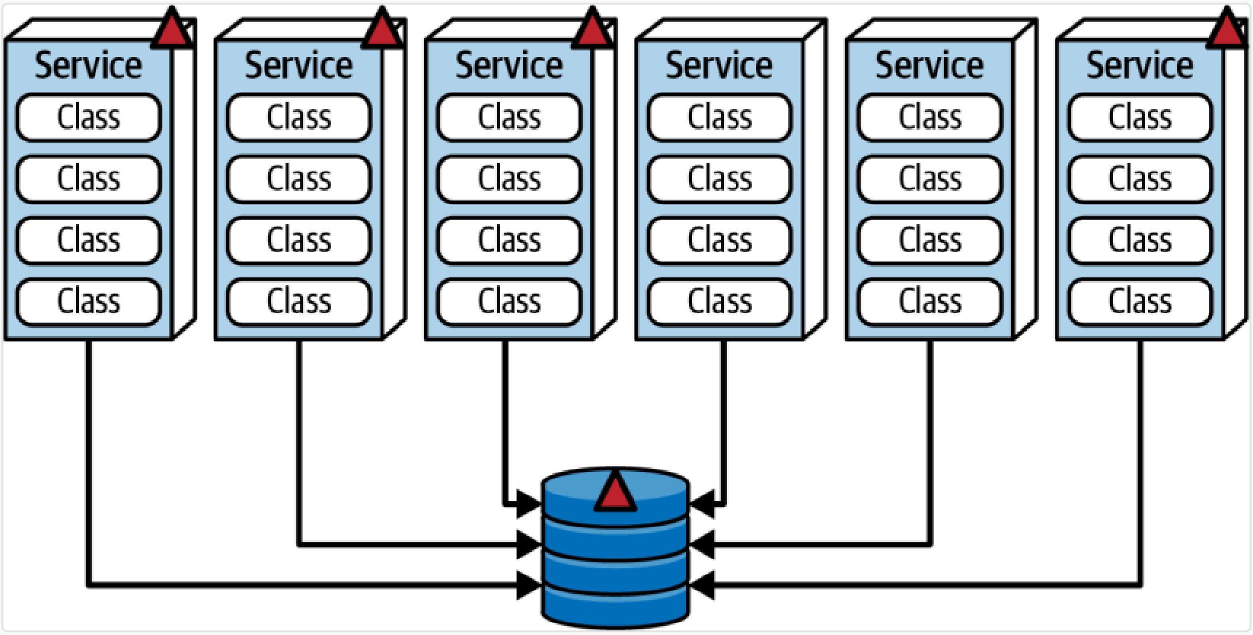
نکته: گاهی لازم است چند تراکنش از مرز سرویس‌ها عبور کنند؛ اما اگر این ویژگیِ غالبِ معماری باشد، به احتمال زیاد میکروسرویس‌ها انتخاب درستی نیست!

مدیریت تراکنش‌ها در میکروسرویس‌ها پیچیده است و می‌توان بسیار عمیق‌تر به آن پرداخت. ما هشت الگوی تراکنشیِ `Saga` را برای حل سناریوهای گوناگون شناسایی کرده‌ایم که می‌توانید آن‌ها را در فصل ۱۲ کتاب دیگرمان، **Software Architecture: The Hard Parts** (O'Reilly، ۲۰۲۱) بیابید؛ کتابی که با همکاری Pramod Sadalage و Zhamak Dehghani نوشته‌ایم.

توپولوژی‌های داده

همان‌طور که در سراسر این فصل گفتیم، داده نقشی حیاتی در معماری‌های میکروسرویس ایفا می‌کند. در واقع، میکروسرویس‌ها تنها سبک معماری‌ای است که معماران را **ملزم** به شکستنِ داده‌ها می‌کند. هرچند در دیگر معماری‌های توزیع‌شده استفاده از پایگاه‌دادهٔ تک‌پارچه **ممکن** است (هرچند همیشه کارآمد نیست)، در میکروسرویس‌ها این کار اصلاً گزینه‌ای در دسترس نیست. پایگاه‌داده‌های حوزه‌ای `(domain databases)` نیز چنین گزینه‌ای نیستند؛ موضوعی که در بخش «توپولوژی‌های داده» در فصل ۱۴ و نیز بخش «توپولوژی‌های داده» در فصل ۱۵ بررسی کردیم. دلیل این امر ماهیت ریزدانهٔ میکروسرویس‌ها، زمینهٔ محدود و شمار زیاد سرویس‌ها در بیشتر اکوسیستم‌های میکروسرویس است.

برای اینکه نشان دهیم چرا پایگاه‌دادهٔ تک‌پارچه در میکروسرویس‌ها شدنی نیست، سناریویی را در نظر بگیرید که در آن ۶۰ سرویس از یک پایگاه‌دادهٔ واحد استفاده می‌کنند. نخستین مشکلی که پیش می‌آید به کنترل تغییر `(change control)` در معماری مربوط است. چنان‌که شکل ۱۸-۱۳ نشان می‌دهد، تغییر ساختار آن پایگاه‌داده (مثلاً تغییر نام یک ستون یا حذف یک جدول) مستلزم تغییرات متناظر در هر ۶۰ سرویسی است که از آن داده استفاده می‌کنند. تصور کنید بخواهید نگهداری، آزمون و انتشار پنج دوجین سرویسِ جداگانه مستقرشده را هماهنگ کنید و همزمان تغییرات پایگاه‌داده را نیز منتشر کنید! این کار، دست‌کم، طاقت‌فرسا خواهد بود و به احتمال زیاد به فاجعه می‌انجامد.



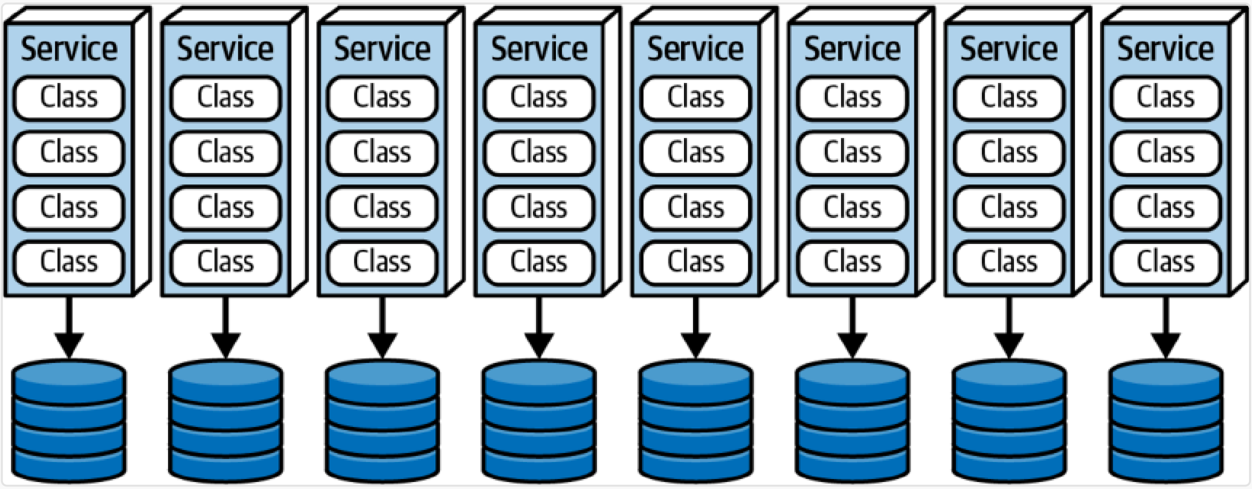
شکل ۱۸-۱۳. کنترل تغییر با یک پایگاه‌دادهٔ تک‌پارچه کاری بسیار دشوار است

شاید بزرگ‌ترین مشکلِ ترکیب توپولوژی دادهٔ تک‌پارچه با میکروسرویس‌ها این باشد که کل مفهوم زمینهٔ محدودِ فیزیکی را از میان می‌برد. به یاد بیاورید که زمینهٔ محدود **همهٔ** کارکردهای لازم برای اجرای یک کارکرد کسب‌وکاری یا زیرحوزهٔ مشخص را در بر می‌گیرد، **از جمله پایگاه‌داده و ساختارهای دادهٔ متناظر**. اگر همهٔ سرویس‌ها از یک پایگاه‌داده و ساختارهای دادهٔ یکسان استفاده کنند، زمینهٔ محدود از بین می‌رود.

مشکل دیگرِ دادهٔ تک‌پارچه به مقیاس‌پذیری و کشسانی مربوط است. هرچند بسیاری از ابزارها و محصولات عملیاتی بار هم‌زمان را به‌طور خودکار پایش می‌کنند و برای پاسخ به افزایش بار، تعداد نمونه‌های سرویس را تنظیم می‌کنند، پایگاه‌داده‌های چندانی نیستند که به همان نسبت مقیاس‌پذیر باشند. این عدم توازن می‌تواند به مشکلات کلی در پاسخ‌گویی `(responsiveness)` و پایان مهلت زمانیِ `(timeout)` درخواست‌ها بینجامد. مدیریت اتصال‌های پایگاه‌داده `(database connections)` که معمولاً در هر **نمونهٔ** سرویس قرار دارند، دغدغهٔ دیگری است. با افزایش تعداد سرویس‌ها و نمونه‌های سرویس، اتصال‌های در دسترسِ پایگاه‌داده به‌سرعت ته می‌کشد و این خود به انتظار بیشتر برای اتصال و پایان مهلت زمانیِ درخواست‌ها می‌انجامد.

سرانجام، اگر پایگاه‌داده به دلیل از کار افتادن، نگهداری برنامه‌ریزی‌شده یا تهیهٔ نسخهٔ پشتیبان `(backup)` در دسترس نباشد، کل اکوسیستم میکروسرویس از کار می‌افتد؛ همان‌طور که هر سبک معماری دیگری که از پایگاه‌دادهٔ تک‌پارچه استفاده کند از کار می‌افتد. توپولوژی پایگاه‌دادهٔ حوزه‌ای `(domain-based database topology)` نیز، هرچند نه به این شدت، ممکن است از همان مشکلاتِ مقیاس‌پذیری، مدیریت مخزن اتصال پایگاه‌داده `(database connection` `pool)`، در دسترس‌بودن `(availability)` و تحمل خطا `(fault tolerance)` رنج ببرد.

به این دلایل، توپولوژی استاندارد پایگاه‌داده برای میکروسرویس‌ها الگوی **پایگاه‌داده به ازای هر سرویس** `(Database-` `per-Service)` است. در این توپولوژی پایگاه‌داده، هر میکروسرویس مالک داده‌های خود است و این داده‌ها به شکل جدول‌هایی درون یک پایگاه‌داده یا طرح‌وارهٔ جداگانه نگهداری می‌شوند؛ چنان‌که در شکل ۱۸-۱۴ نشان داده شده است.

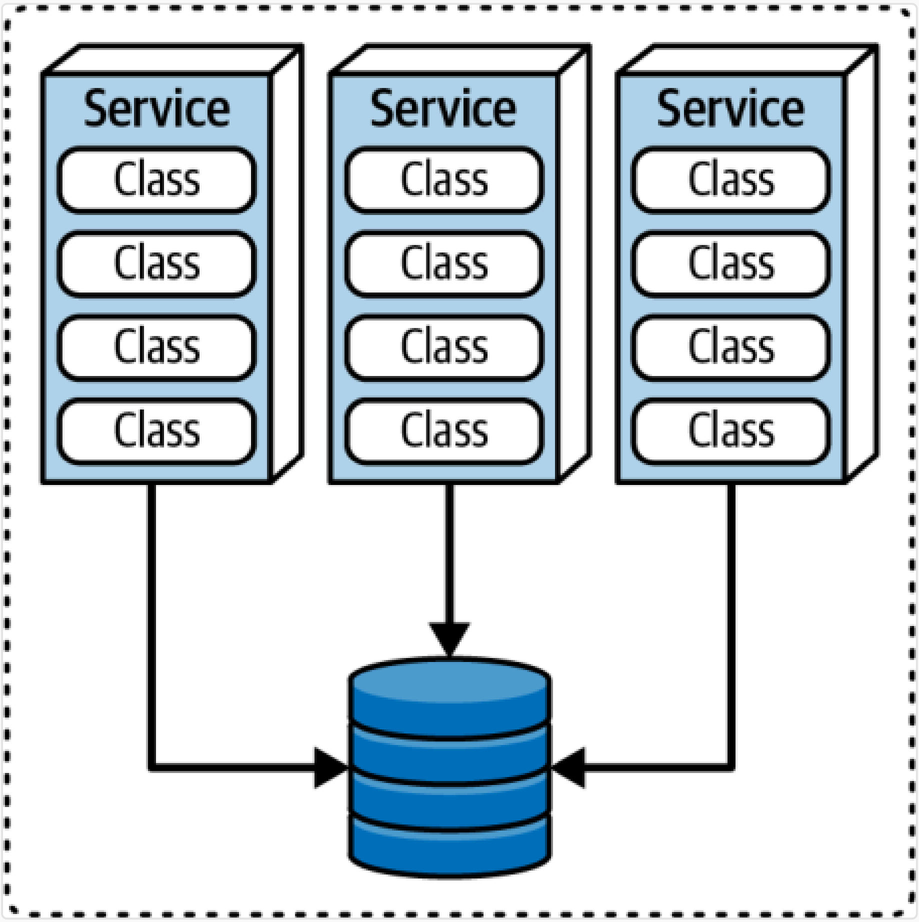


شکل ۱۸-۱۴. الگوی پایگاه‌داده به ازای هر سرویس، توپولوژی رایج پایگاه‌داده در میکروسرویس‌هاست

این توپولوژی زمینهٔ محدود را حفظ می‌کند و کنترل تغییر را برای معماران آسان‌تر می‌سازد. از آنجا که سرویس‌های دیگری که به داده نیاز دارند باید آن را از راه نوعی قرارداد از سرویسِ مالک `(owning service)` درخواست کنند، از ساختار داخلی داده جدا می‌مانند. تغییر ساختار پایگاه‌داده تنها بر سرویسِ مالک درون همان زمینهٔ محدود اثر می‌گذارد. این امر به معماران آزادی می‌دهد که نوع پایگاه‌داده را (مثلاً از پایگاه‌دادهٔ رابطه‌ای به پایگاه‌دادهٔ سندمحور `(document` `database)`) تغییر دهند، بی‌آنکه بر سرویس‌های دیگر اثر بگذارند.

توپولوژی پایگاه‌داده به ازای هر سرویس همچنین مقیاس‌پذیری، کشسانی، در دسترس‌بودن و تحمل خطای عالی‌ای فراهم می‌کند؛ ویژگی‌های معماری‌ای که همگی در توپولوژی‌های پایگاه‌دادهٔ تک‌پارچه یا حوزه‌ای آسیب می‌بینند. افزون بر این، مدیریت اتصال‌ها به پایگاه‌داده درون یک زمینهٔ محدود بسیار آسان‌تر از مدیریت آن‌ها در یک پایگاه‌دادهٔ تک‌پارچه یا حوزه‌ای است.

هرچند توپولوژی پایگاه‌داده به ازای هر سرویس در میکروسرویس‌ها کاربرد گسترده‌ای دارد، بی‌عیب نیست. برای مثال، اگر دو یا چند سرویس در یک جدول پایگاه‌داده بنویسند چه؟ اگر سرویسی بیرون از زمینهٔ محدود، به دلایل کارایی، **ناچار** باشد مستقیماً از پایگاه‌داده پرس‌وجو کند چه؟ در این موارد (که نسبتاً رایج هم هستند)، ممکن است چند سرویس از یک پایگاه‌دادهٔ مشترک استفاده کنند؛ چنان‌که در شکل ۱۸-۱۵ نشان داده شده است. توصیهٔ ما این است که بیش از پنج یا شش سرویس از یک پایگاه‌داده (یا طرح‌واره) مشترک استفاده نکنند. اگر تعداد از این بیشتر شود، با همان مشکلاتِ کنترل تغییر، مقیاس‌پذیری، کشسانی، در دسترس‌بودن، تحمل خطا و مانند آن روبه‌رو خواهید شد.



شکل ۱۸-۱۵. اشتراک داده میان چند میکروسرویس ممکن است

کادری که در شکل ۱۸-۱۵ دور سرویس‌ها و پایگاه‌داده کشیده شده، نشان‌دهندهٔ زمینهٔ محدود است. اینکه سرویس‌ها از یک پایگاه‌دادهٔ مشترک استفاده می‌کنند به این معنا نیست که زمینهٔ محدودی وجود ندارد؛ فقط به این معناست که معمار یک زمینهٔ محدودِ **گسترده‌تر** شکل داده است. برای مثال، ممکن است دلایل موجهی برای شکستنِ پردازش پرداخت به انواع مختلف پرداخت (مانند کارت اعتباری، کارت هدیه، `PayPal`، امتیازهای پاداش و مانند آن) وجود

داشته باشد، اما این سرویس‌های مجزا همچنان باید به داده‌های یکسانی دسترسی داشته باشند و آن‌ها را به‌روزرسانی کنند. به همین ترتیب، معمار ممکن است یک سرویس ارسال کالای واحد را به چند سرویسِ جداگانه مستقرشده، یکی برای هر روش ارسال، بشکند؛ اما همهٔ این سرویس‌ها همچنان به دسترسی به داده‌های یکسان و به‌روزرسانیِ آن‌ها نیاز خواهند داشت.

مصالحهٔ اصلیِ اشتراک داده میان میکروسرویس‌ها درون یک زمینهٔ محدودِ گسترده‌تر، کنترل تغییرات پایگاه‌داده است. وقتی طرح‌وارهٔ پایگاه‌داده تغییر می‌کند، معمار اکنون باید تغییر و استقرار چند سرویس را هماهنگ کند؛ و همین تغییرات پایگاه‌داده را پرمخاطره‌تر و کم‌چابک‌تر می‌سازد. بسته به مسئلهٔ کسب‌وکار و شرایط، ممکن است پیامدهای منفی‌ای نیز برای مقیاس‌پذیری، کشسانی و تحمل خطا در پی داشته باشد.

ملاحظات ابری

هرچند میکروسرویس‌ها را بی‌تردید می‌توان در سیستم‌های درون‌سازمانی `(on-prem)` مستقر کرد، به‌ویژه با محبوبیت پلتفرم‌های ارکستراسیون سرویس `(service orchestration platforms)` مانند `Kubernetes` و `Cloud Foundry`، این سبک معماری برای استقرارهای ابری بسیار مناسب است؛ تا جایی که میکروسرویس‌ها را گاهی معماریِ «بومیِ ابر» `(cloud-native)` می‌نامند. تأمین منابع بر حسب تقاضا `(on-demand provisioning)` برای ماشین‌های مجازی، کانتینرها و پایگاه‌داده‌ها، در کنار رویکرد سرویس‌محوری که در محیط‌های ابری (مانند `AWS`) دیده می‌شود، به‌خوبی با سبک معماری میکروسرویس‌ها سازگار است.

خوانندهٔ دقیق ممکن است بپرسد چرا «بدون سرور» `(serverless)` را به‌عنوان یک سبک معماری نیاورده‌ایم. **بدون سرور** به مدلی از رایانش ابری اشاره دارد که در آن توابع بنا به درخواست فعال می‌شوند و منابع ماشینیِ لازم بر حسب تقاضا تخصیص می‌یابد. توابع بدون سرور مصنوعاتی مانند `AWS Lambda`، `Google Cloud Functions` و `Azure Functions` را شامل می‌شوند. به باور ما، بدون سرور یک سبک معماری نیست، بلکه یک **مدل استقرار** `(deployment model)` برای سبک معماری میکروسرویس‌هاست.

به یاد بیاورید که پیش‌تر در این فصل، **میکروسرویس** را واحدی نرم‌افزاری، تک‌منظوره و جداگانه مستقرشونده تعریف کردیم که **یک کار** را واقعاً خوب انجام می‌دهد (و پیشوند **میکرو** از همین‌جا آمده است). از این رو، میکروسرویس‌ها معمولاً نسبت به سرویس‌ها در دیگر سبک‌های معماری ریزدانه‌ترند. این تعریف در اصل یک **تابع بدون سرور** `(serverless function)` را هم توصیف می‌کند و به همین دلیل، بدون سرور را بخشی از میکروسرویس‌ها می‌دانیم.

با این حال، میکروسرویس‌ها در محیط‌های ابری **ناچار نیستند** به شکل توابع بدون سرور مستقر شوند؛ می‌توان آن‌ها را به همان سادگی به شکل سرویس‌های کانتینری `(containerized services)` نیز مستقر کرد. بیشتر فروشندگان خدمات ابری `Kubernetes` (یا شکلی از پلتفرم `Kubernetes`) را پذیرفته‌اند و به توسعه‌دهندگان امکان می‌دهند میکروسرویس‌های کانتینری را به همان آسانیِ میکروسرویس‌های بدون سرور مستقر کنند.

ریسک‌های رایج

یکی از بزرگ‌ترین ریسک‌ها در میکروسرویس‌ها، ساختنِ سرویس‌هایی است که **بیش از حد** کوچک‌اند. در سال ۲۰۱۶، یکی از ما نویسندگان `(Mark)` نام **دانه‌های شن** `(Grains of Sand)` را برای ضدالگوی ساختنِ سرویس‌های بیش از حد ریزدانه برگزید؛ سرویس‌هایی درست مانند دانه‌های شن در ساحل. همان‌طور که پیش‌تر گفتیم، **میکرو** در میکروسرویس‌ها به **کاری** که سرویس انجام می‌دهد اشاره دارد، نه به اینکه سرویس چقدر **بزرگ** است. دانه‌بندی سرویس‌ها آن‌قدر در میکروسرویس‌ها اهمیت دارد که فصلی کامل (فصل ۷) از کتاب دیگرمان، **Software Architecture: The Hard Parts**، را به آن اختصاص داده‌ایم.

ریسک رایج دیگر در میکروسرویس‌ها، پدید آوردنِ ارتباط بیش از حد میان سرویس‌هاست. ماهیت ریزدانهٔ میکروسرویس‌ها، در کنار زمینه‌های محدودِ تنگ، ایجاب می‌کند که سرویس‌های یک اکوسیستم میکروسرویس ناگزیر با یکدیگر ارتباط برقرار کنند. این ارتباط ممکن است ناشی از پردازش گردش‌کار باشد (مانند کرئوگرافی، یا `AWS Step Functions` برای میکروسرویس‌های بدون سرور) یا ناشی از نیاز به داده‌های سرویسی دیگر (به دلیل زمینهٔ محدودی که هر سرویس و داده‌هایش را در بر می‌گیرد). دلیلش هرچه باشد، مراقب باشید کوپلینگ پویا `(dynamic coupling)` و ارتباط میان‌سرویسی `(interservice communication)` **بیش از حد** نشود. باز هم، این وضعیت اغلب نتیجهٔ ساختنِ سرویس‌های بیش از حد ریزدانه است و می‌توان آن را با ادغام سرویس‌ها در میکروسرویس‌هایی درشت‌دانه‌تر `(coarser-grained)` برطرف کرد.

ریسک دیگر در میکروسرویس‌ها، زیاده‌روی در اشتراک داده است. هرچند گفتیم که اشتراک داده در میکروسرویس‌ها ممکن (و گاه ضروری) است، اشتراک بیش از حد داده، کنترل تغییر، مقیاس‌پذیری، تحمل خطا و چابکیِ کلیِ سیستم را به خطر می‌اندازد؛ یعنی همان چیزهایی که میکروسرویس‌ها در آن‌ها واقعاً خوب عمل می‌کند (نگاه کنید به بخش «ویژگی‌های سبک» در همین فصل). بدانید چه زمانی اشتراک داده ضروری است و چه زمانی باید آن را با ادغام سرویس‌ها `(service consolidation)` اصلاح کرد.

ریسک پایانی، که در سبک معماری میکروسرویس‌ها اغلب نادیده گرفته می‌شود، بازاستفادهٔ کد و اشتراک کارکردهاست. بازاستفادهٔ کد `(code reuse)` بخش ضروریِ توسعهٔ نرم‌افزار است. اما بازاستفادهٔ کد و کارکرد مستقیماً با اصول میکروسرویس‌ها در تضاد است؛ به همین دلیل، این معماری را همان‌طور که پیش‌تر در این فصل گفتیم، معماری «بدون اشتراک» می‌نامند. وقتی معمار کارکردهای مشترک را از طریق کتابخانه‌های سفارشی `(custom libraries)` (مانند فایل‌های `JAR` یا `DLL`) میان سرویس‌ها به اشتراک می‌گذارد، بخشی از زمینهٔ محدود فرو می‌پاشد. به بیان دیگر، کدِ بازاستفاده‌شده در چند زمینهٔ محدود پخش می‌شود؛ یعنی همهٔ کارکردهای آن کارکرد یا زیرحوزهٔ مشخص درون زمینهٔ محدودش جای ندارد و در نتیجه، تغییر در کدِ مشترک ممکن است سرویس‌هایی را در زمینه‌های محدودِ دیگر از کار بیندازد. هرچند نسخه‌بندی `(versioning)` به حل این مسئله کمک می‌کند، اشتراک کد به هر حال پیچیدگی چشمگیری به اکوسیستم میکروسرویس می‌افزاید.

حاکمیت

بسیاری از قواعد و فنون حاکمیتی که در میکروسرویس‌ها به کار می‌روند، به ریسک‌های رایجی می‌پردازند که در بخش پیش شرح دادیم. حاکمیت در یک معماری میکروسرویس بیش از هر چیز به پرهیز از فرسایش ساختاری `(structural decay)` مربوط است.

پیش از هر چیز، معماران باید حاکمیت را برای پایش و کنترل میزان کوپلینگ ایستا `(static coupling)` و پویا میان سرویس‌ها به کار گیرند. از فصل ۷ به یاد بیاورید که کوپلینگ ایستا وقتی رخ می‌دهد که میکروسرویس‌ها از کتابخانه‌های سفارشی یا کتابخانه‌های شخص ثالث `(third-party)` مشترک استفاده کنند؛ و نیز به شکل قراردادها، هنگامی که سرویس‌ها باید با هم ارتباط برقرار کنند. قراردادها اهمیت ویژه‌ای دارند: هرچند معماران می‌توانند با پروتکل‌های ارتباطیِ ناهم‌گام، سرویس‌ها را به‌صورت **پویا** از هم جدا کنند، این سرویس‌ها ممکن است همچنان از راه قراردادی که میانشان به کار می‌رود (صرف‌نظر از نوع ارتباط) به‌صورت **ایستا** کوپل باشند.

فهرست اجزای نرم‌افزار `(software bill of materials)`، اسکریپت‌های استقرار `(deployment scripts)` و ابزارهای مدیریت وابستگی `(dependency-management tools)` می‌توانند به معماران کمک کنند شمار مصنوعاتی را که میان سرویس‌ها مشترک است بهتر بشناسند و بر آن حاکمیت کنند. هرچند نمی‌توانیم دقیقاً تجویز کنیم چه میزان کوپلینگ ایستا **بیش از حد** است، توصیه می‌کنیم برای به حداقل رساندنِ کوپلینگ میان سرویس‌ها تلاش کنید.

حاکمیت بر کوپلینگ پویا بسیار دشوارتر از حاکمیت بر کوپلینگ ایستاست. گردآوریِ سنجه‌های مناسب به خلاقیت و یکدستی نیاز دارد. یکی از فنون رایج حاکمیتی، استفاده از لاگ‌ها برای شناسایی فراخوانی‌ها به سرویس‌های دیگر است. وقتی سرویس‌ها سرویس‌های داخلی یا شخص ثالث را فراخوانی می‌کنند، این تعامل‌ها را همراه با اطلاعاتی دربارهٔ اینکه کدام سرویس فراخوانی شده، از چه پروتکلی استفاده شده و مانند آن ثبت می‌کنند. معماران می‌توانند این اطلاعات را از طریق توابع برازش `(fitness functions)` تحلیل کنند تا سطح کوپلینگ پویا را در سراسر اکوسیستم میکروسرویس بهتر بشناسند. با این حال، این رویکرد به حاکمیتی دقیق نیاز دارد تا اطمینان حاصل شود که هر سرویس این اطلاعات را به شیوه‌ای یکدست از طریق ثبت وقایع در اختیار می‌گذارد. یک کتابخانهٔ سفارشی (مانند یک فایل `JAR` یا `DLL`) یکی از راه‌های فراهم کردنِ یک `API` یکدست است که همهٔ سرویس‌ها می‌توانند در زمان کامپایل `(compile time)` به آن متصل شوند و از آن برای تضمین ثبت وقایعِ یکدست استفاده کنند.

راه دیگر برای گردآوریِ سنجه‌های کوپلینگ پویا در میکروسرویس‌ها، استفاده از مدخل‌های رجیستری `(registry entries)` است. هر بار که نخستین نمونهٔ یک سرویس راه‌اندازی می‌شود، آن سرویس فراخوانی‌های میان‌سرویسیِ خود را از راه نوعی قرارداد (مانند `JSON`) در یک سرویس پیکربندی سفارشی یا یک سرور پیکربندی `(configuration server)`، مانند `Apache ZooKeeper`، ثبت می‌کند. سپس معمار می‌تواند از سرور پیکربندی پرس‌وجو کند تا نقشه‌ای از همهٔ فراخوانی‌های میان‌سرویسی `(interservice calls)` در سراسر اکوسیستم میکروسرویس به دست آورد و از آن برای حاکمیت و کنترل میزان ارتباط میان سرویس‌ها استفاده کند.

ملاحظاتِ توپولوژی تیم

از آنجا که معماری‌های میکروسرویس بر پایهٔ حوزه افراز شده‌اند، وقتی تیم‌ها نیز بر پایهٔ حوزه هم‌راستا شده باشند (مانند تیم‌های چندتخصصی `(cross-functional teams)` با تخصص‌های ویژه) بهترین عملکرد را دارند. وقتی نیازمندی‌ای حوزه‌محور پیش می‌آید، یک تیم چندتخصصیِ متمرکز بر حوزه می‌تواند درون یک سرویس حوزه‌ای مشخص، به‌طور جمعی روی آن قابلیت کار کند، بی‌آنکه در کار تیم‌ها یا سرویس‌های دیگر اختلال ایجاد کند. در مقابل، تیم‌های افرازشده به‌صورت فنی (مانند تیم‌های رابط کاربری، تیم‌های سمت سرور، تیم‌های پایگاه‌داده و مانند آن) به دلیل افراز حوزه‌ایِ این سبک معماری، با آن سازگاری خوبی ندارند. سپردنِ نیازمندی‌های حوزه‌محور به تیمی که بر پایهٔ ملاحظات فنی سازمان یافته، سطحی از ارتباط و همکاریِ میان‌تیمی `(interteam communication)` را می‌طلبد که در بیشتر سازمان‌ها دشوار از آب درمی‌آید.

در ادامه نکاتی آمده است که معماران باید دربارهٔ هم‌راستاییِ یک معماری میکروسرویس با توپولوژی‌های تیمِ مشخص که در بخش «توپولوژی‌های تیم و معماری» در فصل ۹ معرفی شدند در نظر بگیرند:

تیم‌های هم‌راستا با جریان **`(Stream-aligned teams)`**: اگر مرزهای حوزه به‌درستی هم‌راستا شده باشند، تیم‌های هم‌راستا با جریان به‌خوبی با این سبک معماری کار می‌کنند؛ به‌ویژه اگر جریان‌هایشان بر یک حوزهٔ مشخص متمرکز باشد. اما برای تیم‌هایی که جریان‌هایشان از چند زمینهٔ محدود و چند سرویس، بیرون از یک زیرحوزه یا حوزهٔ مشخص، عبور می‌کند، معماری میکروسرویس‌ها چالش‌برانگیزتر می‌شود. در این حالت، توصیه می‌کنیم زمینه‌های محدود و دانه‌بندیِ میکروسرویس‌ها را تحلیل کنید و یا آن‌ها را با جریان‌ها از نو هم‌راستا کنید، یا سبک معماری دیگری برگزینید.

تیم‌های توانمندساز **`(Enabling teams)`**: تیم‌های توانمندساز در یک معماری میکروسرویس زمانی بیشترین اثربخشی را دارند که بتوانند از سرویس‌های مشترک برای دغدغه‌های تخصصی یا میان‌بُرشی استفاده کنند. به دلیل ماژولاریتیِ `(modularity)` بالای میکروسرویس‌ها، تیم‌های توانمندساز می‌توانند مستقل از تیم‌های هم‌راستا با جریان کار کنند و کارکردهای تخصصی و مشترکِ بیشتری فراهم آورند، بی‌آنکه سد راه دیگران شوند. آن‌ها می‌توانند با همکاری تیم‌های پلتفرم، در ساختنِ مؤلفه‌های `sidecar` که مش سرویس را تشکیل می‌دهند نیز کمک کنند (نگاه کنید به بخش «بازاستفادهٔ عملیاتی» در همین فصل).

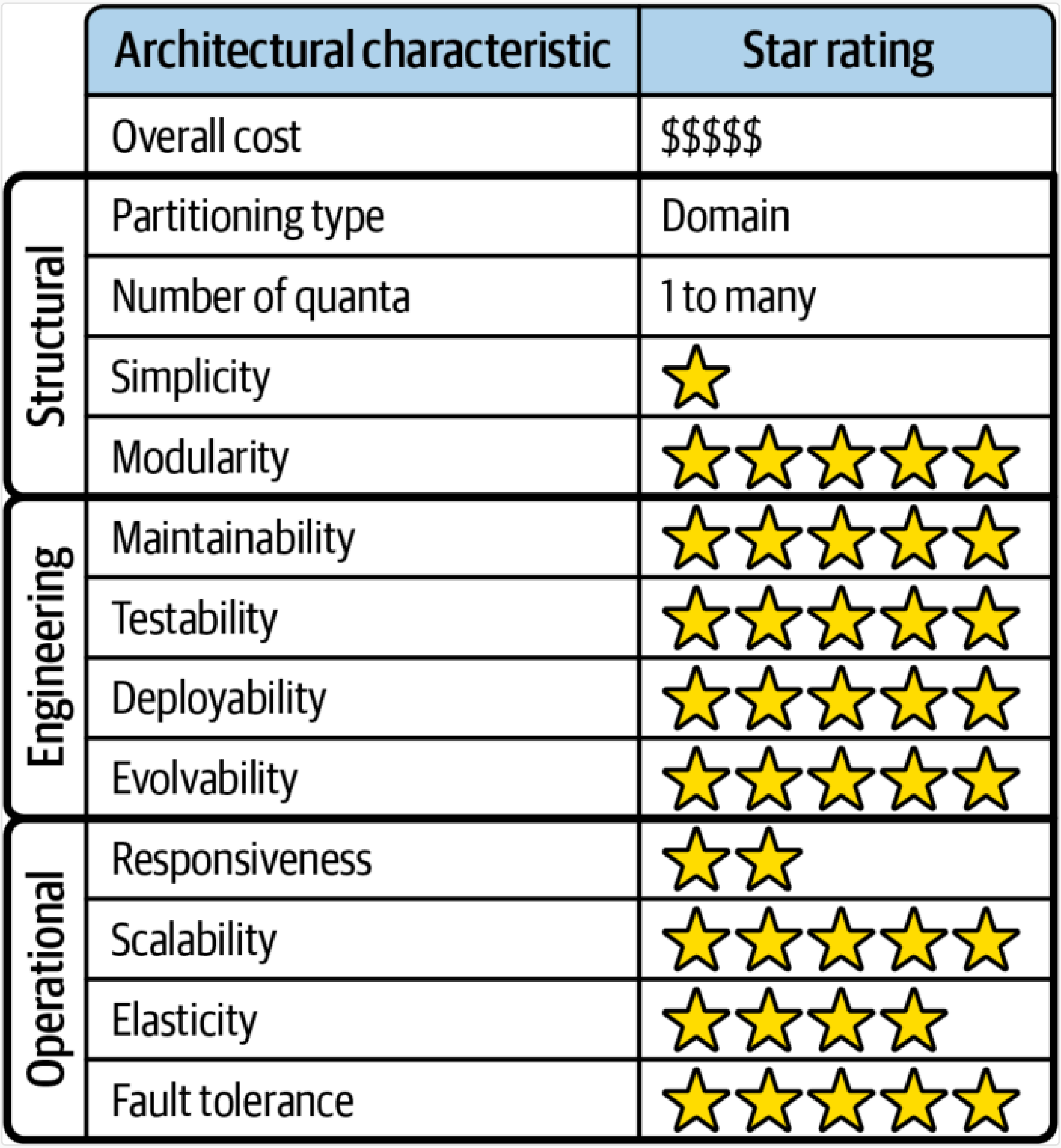
تیم‌های زیرسیستم پیچیده **`(Complicated-subsystem teams)`**: تیم‌های زیرسیستم پیچیده می‌توانند از ماژولاریتیِ سطح سرویس در این سبک معماری بهره بگیرند تا بر پردازش‌های پیچیدهٔ یک حوزه یا زیرحوزه تمرکز کنند و از دیگر اعضای تیم (و سرویس‌ها) مستقل بمانند.

تیم‌های پلتفرم **`(Platform teams)`**: ماژولاریتیِ بالای میکروسرویس‌ها به تیم‌های هم‌راستا با جریان کمک می‌کند با به‌کارگیریِ ابزارها، سرویس‌ها، `API` ها و وظایف مشترک، از مزایای توپولوژی تیم پلتفرم بهره ببرند. در بسیاری از موارد، تیم‌های پلتفرم (گاه با همکاری تیم‌های توانمندساز) بر ساختن و نگهداریِ کارکردهای عملیاتیِ میان‌بُرشی‌ای تمرکز می‌کنند که در `sidecar` ها (نگاه کنید به بخش «بازاستفادهٔ عملیاتی» در همین فصل) و مش سرویس وجود دارند و به این ترتیب، تیم‌های هم‌راستا با جریان را از این دغدغه‌های عملیاتی آزاد می‌کنند.

ویژگی‌های سبک

سبک معماری میکروسرویس‌ها در مقیاس رتبه‌بندیِ استاندارد ما، که در شکل ۱۸-۱۶ نشان داده شده، چندین رتبهٔ حدی دارد. رتبهٔ یک‌ستاره یعنی آن ویژگی معماریِ مشخص در این معماری به‌خوبی پشتیبانی نمی‌شود، در حالی که رتبهٔ پنج‌ستاره یعنی آن ویژگی معماری یکی از قوی‌ترین جنبه‌های این سبک است. تعریفِ ویژگی‌هایی که در این کارنامه آمده‌اند در فصل ۴ یافت می‌شود.

سبک میکروسرویس‌ها از شیوه‌های مهندسیِ مدرن `(modern engineering practices)` مانند استقرار خودکار `(automated deployment)` و آزمون‌پذیری `(testability)` پشتیبانیِ چشمگیری می‌کند. میکروسرویس‌ها بدون انقلاب `DevOps` و حرکت بی‌وقفه‌اش به سوی خودکارسازیِ دغدغه‌های عملیاتی نمی‌توانست وجود داشته باشد.



	Architectural characteristic	Star rating
	Overall cost	$$$$$
Structural	Partitioning type	Domain
	Number of quanta	1 to many
	Simplicity	★
	Modularity	★★★★★
Engineering	Maintainability	★★★★★
	Testability	★★★★★
	Deployability	★★★★★
	Evolvability	★★★★★
Operational	Responsiveness	★★
	Scalability	★★★★★
	Elasticity	★★★★
	Fault tolerance	★★★★★

شکل ۱۸-۱۶. رتبه‌بندیِ ویژگی‌های میکروسرویس‌ها

ماهیت مستقل، تک‌منظوره و در نتیجه ریزدانهٔ سرویس‌ها در این سبک معماری معمولاً به تحمل خطای بالا می‌انجامد؛ رتبهٔ بالای این ویژگی نیز از همین‌جاست.

دیگر نقاط قوت این معماری، مقیاس‌پذیری، کشسانی و تکامل‌پذیری `(evolvability)` است. برخی از مقیاس‌پذیرترین سیستم‌هایی که تاکنون نوشته شده‌اند، با موفقیت چشمگیری از میکروسرویس‌ها بهره برده‌اند. به همین ترتیب، از آنجا که این سبک به‌شدت به خودکارسازی و یکپارچگیِ هوشمند با عملیات متکی است، معماران می‌توانند پشتیبانی از کشسانی را در آن بگنجانند. از آنجا که این معماری جداسازیِ بالا را در سطحی تدریجی ترجیح می‌دهد، از رویهٔ کسب‌وکاریِ مدرنِ تغییر تکاملی `(evolutionary change)`، حتی در سطح معماری، نیز پشتیبانی

می‌کند. کسب‌وکارهای مدرن سریع حرکت می‌کنند و توسعهٔ نرم‌افزار برای همگام ماندن با آن‌ها به زحمت افتاده است. معماری‌ای که از واحدهای استقرارِ `(deployment units)` بسیار کوچک و به‌شدت جداشده ساخته شده باشد، می‌تواند از آهنگ سریع‌ترِ تغییر پشتیبانی کند.

کارایی اغلب در میکروسرویس‌ها مشکل‌ساز است. معماری‌های توزیع‌شده برای انجام کار باید فراخوانی‌های شبکه‌ایِ زیادی انجام دهند و این کار سربار کاراییِ بالایی دارد. آن‌ها همچنین باید برای تأیید هویت و دسترسی در هر نقطهٔ پایانی، بررسی‌های امنیتی `(security checks)` را فراخوانی کنند که تأخیر بیشتری به همراه دارد. میکروسرویس‌ها از **تأخیر داده** `(data latency)` نیز رنج می‌برد: وقتی درخواستی به هماهنگیِ چند سرویس نیاز دارد، به معنای چند فراخوانیِ پایگاه‌داده است.

به همین دلیل، دنیای میکروسرویس‌ها الگوهای معماریِ فراوانی برای افزایش کارایی دارد؛ از جمله کش‌گذاری و تکثیر هوشمندانهٔ داده برای جلوگیری از فراخوانی‌های شبکه‌ایِ بیش از حد. کارایی دلیل دیگری است که میکروسرویس‌ها اغلب کرئوگرافی را بر ارکستراسیون ترجیح می‌دهند: کوپلینگ کمتر امکان ارتباط سریع‌تر و گلوگاه‌های `(bottlenecks)` کمتری را فراهم می‌کند.

سبک میکروسرویس‌ها قاطعانه یک معماری با افراز حوزه‌ای است که در آن مرز هر سرویس باید با حوزه‌ها متناظر باشد. به لطف زمینهٔ محدود، این سبک متمایزترین کوانتوم‌ها `(quanta)` را در میان همهٔ معماری‌های مدرن دارد؛ از بسیاری جهات، نمونهٔ اعلای همان چیزی است که سنجهٔ کوانتوم ارزیابی می‌کند. فلسفهٔ پیشرانِ جداسازیِ حداکثری دردسرهایی پدید می‌آورد، اما وقتی به‌درستی اجرا شود، مزایای فوق‌العاده‌ای به بار می‌آورد. همانند هر معماری دیگری، معماران باید قواعد را بفهمند تا بتوانند هوشمندانه آن‌ها را بشکنند.

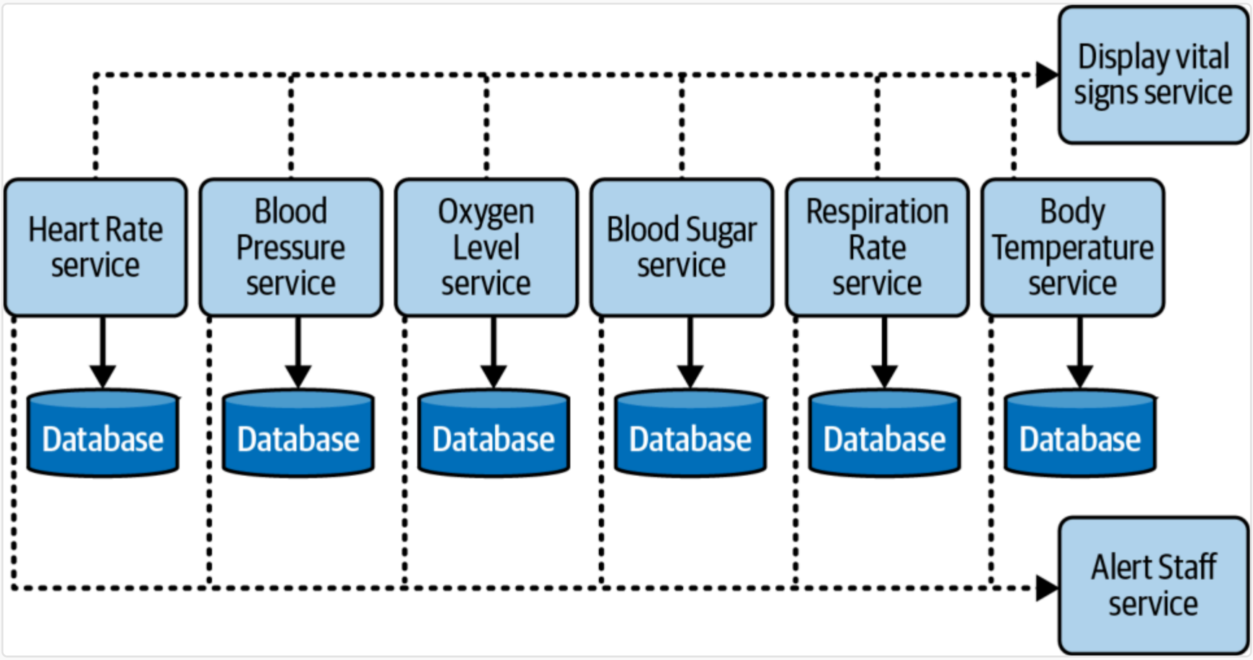
نمونه‌ها و موارد کاربرد

سیستم‌هایی که ماژولاریتیِ کارکردی و داده‌ایِ بالایی دارند، نامزدهای خوبی برای معماری میکروسرویس‌ها هستند. یک مورد کاربرد خوب که قدرت این سبک را نشان می‌دهد، سیستم پایش پزشکی‌ای است که علائم حیاتیِ `(vital signs)` بیمار را پایش می‌کند: ضربان قلب، فشار خون، سطح اکسیژن و مانند آن. هر علامت حیاتی که سیستم پایش می‌کند، کارکردی جداگانه و مستقل است که داده‌های خودش را مدیریت می‌کند؛ و این به‌خوبی با این سبک معماری و مفهوم زمینهٔ محدود سازگار است.

این سیستم پایش بیمار ورودی‌ها را از دستگاه‌های پایش بیمار می‌خواند، علائم حیاتی را ثبت می‌کند، آن‌ها را از نظر هرگونه مشکل یا ناهمخوانی تحلیل می‌کند و در صورت یافتن مشکل، به یک متخصص پزشکی هشدار می‌دهد. هر علامت حیاتی را می‌توان با یک میکروسرویس جداگانه بازنمایی کرد که تا حد زیادی از سرویس‌ها و داده‌های دیگر مستقل است. با این حال، یک استثنا بر این استقلال وجود دارد: هنگامی که یک علامت حیاتیِ مشخص (مانند ضربان قلب) برای تحلیل خود از نظر هشدارها یا مشکلات، به اطلاعات بیشتری از علامت حیاتیِ دیگری (مانند سرویس پایش خواب) نیاز داشته باشد.

شکل ۱۸-۱۷ نشان می‌دهد این سیستم چگونه ممکن است با یک معماری میکروسرویس طراحی شود. توجه کنید که هر علامت حیاتی به شکل یک میکروسرویس جداگانه پیاده‌سازی شده و هر کدام مخزن دادهٔ `(data store)` خودش را برای خوانش‌های علائم حیاتی و داده‌های تاریخی نگه می‌دارد.

کارکرد هشدار، که میان همهٔ سرویس‌های علائم حیاتی مشترک است، با یک **سرویس مشترک** `(shared service)` به نام `Alert Staff` بازنمایی شده است. اگر سرویسی مشکلی در یک خوانش ببیند، این سرویس به پرستار یا پزشک هشدار می‌دهد. هر سرویس آخرین خوانش علائم حیاتی خود را به‌صورت ناهم‌گام به نمایشگری در اتاق بیمار می‌فرستد که با سرویس مشترک `Display Vital Signs` بازنمایی شده است.


Display vital signs service
Heart Rate service
Blood Pressure service
Oxygen Level service
Blood Sugar service
Respiration Rate service
Body Temperature service
Database
Database
Database
Database
Database
Database
Alert Staff service

شکل ۱۸-۱۷. سیستم پایش پزشکیِ بیمار که با معماری میکروسرویس‌ها پیاده‌سازی شده است

این نمونه برخی از نقاط قوت و مزایای معماری میکروسرویس‌ها را به‌روشنی نشان می‌دهد. **تحمل خطا** را در نظر بگیرید: اگر یکی از سرویس‌های علائم حیاتی از کار بیفتد یا پاسخ ندهد، همهٔ سرویس‌های دیگرِ پایش علائم حیاتی کاملاً عملیاتی باقی می‌مانند. این موضوع به‌ویژه در پایش پزشکی اهمیت دارد. ابرقدرت دیگر، **آزمون‌پذیری** است. اگر توسعه‌دهنده‌ای روی سرویس پایش فشار خون کار نگهداری انجام دهد، دامنهٔ آزمون آن‌قدر کوچک است که می‌تواند مطمئن باشد این علامت حیاتیِ مشخص به‌طور کامل آزموده شده و کار نگهداری بر دیگر سرویس‌های علائم حیاتی اثری نگذاشته است. سرانجام، نشانه‌ای از **تکامل‌پذیری** (یکی دیگر از ابرقدرت‌های میکروسرویس‌ها) این است که معمار می‌تواند به‌آسانی پایشگر علامت حیاتیِ دیگری بیفزاید، بی‌آنکه بر سرویس‌های دیگر اثر بگذارد.

در این فصل به بسیاری از جنبه‌های مهم معماری میکروسرویس‌ها پرداختیم و منابع عالیِ فراوانی برای آموختن بیشتر وجود دارد. برای کاوشی عمیق در میکروسرویس‌ها، منابع زیر را توصیه می‌کنیم:

- **Building Microservices**، ویراست دوم، نوشتهٔ Sam Newman (O’Reilly، ۲۰۲۱)
- **Building Micro-Frontends**، ویراست دوم، نوشتهٔ Luca Mezzalira (O’Reilly، ۲۰۲۵)

- **Microservices vs. Service-Oriented Architecture** نوشتهٔ Mark Richards (O’Reilly، ۲۰۱۶)
- **Microservices AntiPatterns and Pitfalls** نوشتهٔ Mark Richards (O’Reilly، ۲۰۱۶)

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Microservices Architecture	معماری میکروسرویس‌ها
Domain-Driven Design (DDD)	طراحی حوزه‌محور (DDD)
Bounded Context	زمینهٔ محدود
Share Nothing (architecture)	(معماری) بدون اشتراک
First Law of Software Architecture	قانون اول معماری نرم‌افزار
Inheritance	وراثت
Composition	ترکیب
Duplication	دوباره‌نویسی
Distributed Architecture	معماری توزیع‌شده
Process (OS)	فرایند
Virtual Machine	ماشین مجازی
Container	کانتینر
Multitenant Infrastructure	زیرساخت چندمستأجری
Application Server	سرور برنامهٔ کاربردی
Operational Reuse	بازاستفادهٔ عملیاتی
Automated Machine Provisioning	تأمین خودکار ماشین‌ها
Network Call	فراخوانی شبکه‌ای
Method Call	فراخوانی متد
Endpoint	نقطهٔ پایانی
Granularity	دانه‌بندی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Subdomain	زیرحوزه
Workflow	گردش‌کار
Domain Partitioning	افراز حوزه‌ای
Big Ball of Distributed Mud	توپ بزرگِ گِلِ توزیع‌شده
Fine-Grained	ریزدانه
Functional Cohesion	انسجام کارکردی
Entity	موجودیت
Choreography	کرئوگرافی
Iteration	تکرار
Data Isolation	جداسازی داده
Entity Trap	تلهٔ موجودیت
Relational Database	پایگاه‌دادهٔ رابطه‌ای
Single Source of Truth	منبع واحد حقیقت
Database Replication	تکثیر پایگاه‌داده
Caching	کش‌گذاری
API Layer	لایهٔ API
API Gateway	دروازهٔ API
Reverse Proxy	پراکسی معکوس
Cross-Cutting Concerns	دغدغه‌های میان‌بُرشی
Naming Service	سرویس نام‌گذاری
Mediator	میانجی
Orchestrator	هماهنگ‌کننده
Technical Partitioning	افراز فنی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Request Routing	مسیریابی درخواست
Monitoring	پایش
Logging	ثبت وقایع
Circuit Breaker	قطع‌کنندهٔ مدار
Sidecar (pattern)	Sidecar (بدون ترجمه؛ نام الگو)
Service Mesh	مش سرویس
Service Plane	صفحهٔ سرویس
Node (graph)	گره
Service Discovery	کشف سرویس
Elasticity	کشسانی
Service Instance	نمونهٔ سرویس
User Interface (UI)	رابط کاربری (UI)
Monolithic Frontend	سمت کاربرِ تک‌پارچه
Micro-Frontends	میکروفرانت‌اندها
Synchronous Communication	ارتباط همگام
Asynchronous Communication	ارتباط ناهمگام
Protocol-Aware Heterogeneous Interoperability	همکنش‌پذیریِ ناهمگونِ آگاه از پروتکل
Protocol-Aware	آگاه از پروتکل
Heterogeneous	ناهمگون
Interoperability	همکنش‌پذیری
Integration Hub	هاب یکپارچه‌سازی
Message Queue	صف پیام
Technology Stack	پشتهٔ فناوری

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Polyglot Environment	محیط چندزبانه
Enforced Heterogeneity	ناهمگونیِ اجباری
Wish List	فهرست علاقه‌مندی‌ها
Orchestration Service	سرویس ارکستراسیون
Error Handling	مدیریت خطا
Front Controller (pattern)	Front Controller (بدون ترجمه؛ نام الگو)
Atomicity	اتمی‌بودن
Dynamic Connascence	هم‌بستگی پویا
Connascence of Values	هم‌بستگی مقادیر
Distributed Transaction	تراکنش توزیع‌شده
Saga Pattern	الگوی Saga
Compensating Transaction Framework	چارچوب تراکنش جبرانی
Pending (state)	وضعیت معلق
Domain Database	پایگاه‌دادهٔ حوزه‌ای
Change Control	کنترل تغییر
Responsiveness	پاسخ‌گویی
Timeout	مهلت زمانی
Database Connection	اتصال پایگاه‌داده
Backup (data)	نسخهٔ پشتیبان
Domain-Based Database Topology	توپولوژی پایگاه‌دادهٔ حوزه‌ای
Database Connection Pool	مخزن اتصال پایگاه‌داده
Availability	در دسترس‌بودن
Fault Tolerance	تحمل خطا

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Database-per-Service	پایگاه‌داده به ازای هر سرویس
Owning Service	سرویسِ مالک
Document Database	پایگاه‌دادهٔ سندمحور
On-Premises (On-Prem)	درون‌سازمانی
Service Orchestration Platform	پلتفرم ارکستراسیون سرویس
Cloud-Native	بومیِ ابر
On-Demand Provisioning	تأمین منابع بر حسب تقاضا
Serverless	بدون سرور
Deployment Model	مدل استقرار
Serverless Function	تابع بدون سرور
Containerized Service	سرویس کانتینری
Grains of Sand (antipattern)	دانه‌های شن
Dynamic Coupling	کوپلینگ پویا
Interservice Communication	ارتباط میان‌سرویسی
Coarse-Grained	درشت‌دانه
Service Consolidation	ادغام سرویس‌ها
Code Reuse	بازاستفادهٔ کد
Custom Library	کتابخانهٔ سفارشی
Versioning	نسخه‌بندی
Structural Decay	فرسایش ساختاری
Static Coupling	کوپلینگ ایستا
Third-Party	شخص ثالث
Software Bill of Materials	فهرست اجزای نرم‌افزار

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Deployment Script	اسکریپت استقرار
Dependency-Management Tool	ابزار مدیریت وابستگی
Fitness Function	تابع برازش
Compile Time	زمان کامپایل
Registry Entry	مدخل رجیستری
Configuration Server	سرور پیکربندی
Interservice Call	فراخوانی میان‌سرویسی
Cross-Functional Team	تیم چندتخصصی
Interteam Communication	ارتباط میان‌تیمی
Stream-Aligned Team	تیم هم‌راستا با جریان
Enabling Team	تیم توانمندساز
Complicated-Subsystem Team	تیم زیرسیستم پیچیده
Platform Team	تیم پلتفرم
Modularity	ماژولاریتی
Modern Engineering Practices	شیوه‌های مهندسیِ مدرن
Automated Deployment	استقرار خودکار
Testability	آزمون‌پذیری
Evolvability	تکامل‌پذیری
Evolutionary Change	تغییر تکاملی
Deployment Unit	واحد استقرار
Security Check	بررسی امنیتی
Data Latency	تأخیر داده
Bottleneck	گلوگاه

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
`Architecture Quantum (pl. Quanta)`	کوانتوم معماری (ج. کوانتوم‌ها)
`Vital Signs`	علائم حیاتی
`Data Store`	مخزن داده
`Shared Service`	سرویس مشترک